Pannon Egyetem
Műszaki Informatikai Kar
Villamosmérnöki és Információs Rendszerek Tanszék
Programtervező informatikus BSc

SZAKDOLGOZAT

Online monopoly típusú játék fejlesztése

Szente Dániel

Témavezető: Ábrahám Gyula

2022

PANNON EGYETEM

MŰSZAKI INFORMATIKAI KAR

Programtervező informatikus BSc szak

Veszprém, 2022. március 23.

SZAKDOLGOZAT TÉMAKIÍRÁS

Szente Dániel
Programtervező informatikus BSc szakos hallgató részére

Online monopoly típusú játék fejlesztése

Témavezető: Ábrahám Gyula, tanársegéd

A feladat leírása:

A hallgató feladata egy online monopoly típusú játék fejlesztése webes technológiák felhasználásával. A játék online jellege miatt kezeli a többjátékos módot úgynevezett szobarendszerben. A szobákhoz a játékosok jelszóval vagy anélkül csatlakozhatnak, és az egy szobába kerülő játékosok alkotnak egy csapatot. A játékmenet logikailag azonos a jól ismert Monopoly társasjátékkal, azonban a grafikai elemek és az egyes szabályok a hallgató saját fejlesztése. Mivel egy összetett, online játékról van szó, így a hallgatónak mind a kliens, mind pedig a szerveroldalt is meg kell terveznie és implementálni.

Feladatkiírás:

- Dolgozza fel a témával kapcsolatos eddigi irodalmat, elérhető implementációkat.
- Határozza meg a szoftverrel szemben támasztott követelményeket.
- Tervezze meg a játékhoz szükséges adatstruktúrát, algoritmusokat és logikát.
- Tervezze meg a rendszer komponenseit, moduljait és azok kapcsolódását.
- Implementálja a játékot.
- Tesztelje a rendszert és szükség esetén végezze el a hibajavításokat.

Dr. Süle Zoltán
egyetemi docens
szakfelelős

Ábrahám Gyula
tanársegéd
témavezető

Hallgatói nyilatkozat

Alulírott Szente Dániel hallgató kijelentem, hogy a dolgozatot a Pannon Egyetem Villamosmérnöki és Információs Rendszerek Tanszékén készítettem a programtervező informatikus végzettség megszerzése érdekében.

Kijelentem, hogy a dolgozatban lévő érdemi rész saját munkám eredménye, az érdemi részen kívül csak a hivatkozott forrásokat (szakirodalom, eszközök stb.) használtam fel.

Tudomásul veszem, hogy a dolgozatban foglalt eredményeket a Pannon Egyetem, valamint a feladatot kiíró szervezeti egység saját céljaira szabadon felhasználhatja.

Dátum: Veszprém , 2022. május 8.



Szente Dániel

Témavezetői nyilatkozat

Alulírott Ábrahám Gyula témavezető kijelentem, hogy a dolgozatot Szente Dániel a Pannon Egyetem Villamosmérnöki és Információs Rendszerek Tanszékén készítette a programtervező informatikus végzettség megszerzése érdekében.

Kijelentem, hogy a dolgozat védésre bocsátását engedélyezem.

Dátum: Veszprém , 2022. május 9.

.....

Ábrahám Gyula

Köszönetnyilvánítás

Köszönöm témavezetőmnek, Ábrahám Gyulának, hogy lehetőséget biztosított számomra a saját témám megvalósításához. Emellett köszönöm neki a hasznos ötleteket, tanácsokat és az idejét. Támogatása és szakértelme rengeteget segített a dolgozat elkészítésében.

Szeretnék kiemelten köszönetet mondani anyukámnak, amiért a tanulmányaim során mindvégig önzetlenül támogatott és hitt bennem, legfőképpen a nehéz időkben. Nélküle ez a dolgozat nem születhetett volna meg.

Végül szeretném megköszönni a családom többi tagjának és a barátaimnak, amiért mindvégig támogattak célom elérésében.

Tartalmi összefoglaló

Szakdolgozatom célja egy online monopoly szerű játék készítse, ahol a játékosok akár négy fős csapatokban tudnak egymással játszani, illetve lehetőségük van szobákat (későbbiekben lobby) létrehozni majd meghívni egymást azokba.

A megvalósítás webes technológiákkal történt. A backendet egy Node.js [1] motort használó Express [2] szerver biztosítja. A kommunikáció websocketeken keresztül történik Socket.io [3] nevű node.js modulon keresztül.

A frontend a jelenleg legpopulárisabb és legkeresettebb JavaScript könyvtár, ami nem más, mint a React JS [4]. A játék egy izometrikus nézetben jelenik meg, így jobb a játék átláthatósága, illetve az esetek többségében a játékosok is jobban kedvelik ilyen játékoknál az izometrikus nézetet. Maga a játék grafikai elemeinek kirenderelése egy canvasra PixiJS [5]-el valósult meg, ami egy HTML5 alkotói motor.

A játék kinézetét illetően letisztult stílusú, ami manapság egyre népszerűbb, és egyre nagyobb játkosbázisnak örvend a stílus.

Kulcsszavak: Monopoly, társasjáték, Node.js, express, Socket.io, ReactJS, izometrikus, PixiJS

Abstract

The aim of my Thesis is the creation of a simple Monopoly-like game, in which, at most four players can play at a time with each other, while also having the ability to create lobbies, where they can invite one another.

The implementation was made by web technologies. The backend is provided by an Express server, using a Node.js engine. The communication takes place through websockets, via a module named Socket.io.

The frontend is the most popular JavaScript library nowadays called React JS. The game is displayed in an izometric view, this way, it is easier to view and more transparent. Also, in most cases, players prefer this view in such games. The graphics elements are rendered by a HTML5 Creation Engine called PixiJS.

Regarding the visuals of the game, it was made in modern and clean design, which has gained plenty of popularity recently, and boasting an ever-growing playerbase.

Keywords: Monopoly, board game, Node.js, express, Socket.io, ReactJs, isometric, PixiJS

Tartalomjegyzék

1. Bevezetés **10**
1.1. Mi is az a Monopoly? . . . 10
1.2. Az ötlet . . . 11
1.3. Hasonló játékok . . . 11
1.4. Elképzelés a játékról . . . 12

2. Irodalmi áttekintés **13**

3. Technológiai áttekintés **14**
3.1. ReactJS . . . 14
3.2. Express . . . 15
3.3. Socket.IO . . . 15
3.4. PixiJS . . . 16
3.5. Material Icons . . . 16
3.6. Mantine UI . . . 16
3.7. Tailwind CSS . . . 16
3.8. i18next . . . 17

4. Feladat bemutatása **18**
4.1. Áttekintés . . . 18

5. Tervezés és modellezés **21**
5.1. Színek . . . 21
5.2. Oldaltérkép . . . 22
5.3. Architektúra . . . 23
5.4. Látványtervek . . . 24
5.4.1. Főoldal . . . 25
5.4.2. Nyelvválasztó menü . . . 26
5.4.3. Szobák . . . 26

5.4.4. Új szoba létrehozása 27
5.4.5. Szoba 28
5.4.6. Játék 29
5.4.7. Ranglista 30

6. Implementáció és tesztek **31**
6.1. Implementáció 31
6.1.1. Szobarendszer 31
6.1.1.1. Főoldal 31
6.1.1.2. Szobák 33
6.1.1.3. Új szoba létrehozása 34
6.1.1.4. Szoba 35
6.1.1.5. Nyelvválasztó menü 36
6.1.1.6. Websocket események 36
6.1.2. Játék 37
6.1.2.1. Játék felépítése 37
6.1.2.2. Mezők generálása 39
6.1.2.3. Dobókockák 40
6.1.2.4. Vásárlás és fejlesztés 41
6.1.2.5. Eladás 43
6.1.2.6. Csőd 45
6.1.2.7. Köridő lejárása 46
6.1.2.8. Normál mező 46
6.1.2.9. Múzeumok 48
6.1.2.10. Karantén 49
6.1.2.11. Talár 51
6.1.2.12. Erasmus 53
6.1.2.13. Adó 54
6.1.2.14. Esély 55
6.1.2.15. Websocket események 56
6.1.2.16. Grafikai elemek 59
6.2. Tesztek 60
6.2.1. Első tesztelés 60
6.2.2. Második tesztelés 60
6.2.3. Harmadik tesztelés 60

6.2.4. Visszajelzések 61
6.2.4.1. Mennyire találja jó ötletnek a játékot? 61
6.2.4.2. Mennyire könnyen és egyértelműen kezelhető a játék? 61
6.2.4.3. Mennyire érezte jól magát játék közben? 62
6.2.4.4. Hány perces játékot tartja optimálisnak? 62
6.2.4.5. Találkozott buggal (hibával) a játék során? 63
6.2.4.6. Hiányolja a hangokat, illetve háttérzenét a játék során? 63
6.2.4.7. Ön szerint, ha ez a játék megjelenne, játszana vele? 64
6.2.4.8. Ajánlaná ismerőseinek a játékot? 64
6.2.4.9. Ha bármilyen észrevétele vagy ötlete van a játékkal kapcsolatban kérem írja le. 64

7. Szoftver **66**
7.1. A szoftver telepítése és futtatása 66
7.1.1. Szükséges programok 66
7.1.2. Telepítés és futtatás 66

8. További lehetőségek/továbbfejlesztési tervek **67**
8.1. Authentikáció 67
8.2. Lokális multiplayer 67
8.3. Hangok és zene 68
8.4. Esély kártyák bővítése 68
8.5. Fizetős tartalom 68
8.6. Matchmaking 68
8.7. Telefon, tablet támogatás 69
8.8. Alkalmazás és applikáció 69
8.9. Más nyelvek támogatása 69

9. Összefoglalás **70**

Irodalomjegyzék **72**

10.Mellékletek **73**
10.0.0.1. Beadott fájlok 73

1. fejezet

Bevezetés

Ebben a fejezetben megtudhatjuk, hogy mi is az a monopoly, honnan ered, illetve, hogy mekkora népszerűségnek örvend. Azt követően az ötlet megszületéséről olvashatunk. Hasonló játékokat is megismerhetünk, a gyengébb megvalósításoktól egészen az Ubisoft által kiadott sikerjátékig. Majd végül egy rövid elképzelésbe tekinthetünk be, hogy mik az előre kitűzött elvárások.

1.1. Mi is az a Monopoly?

Aki még nem ismerné a Monopolyt, az egy társasjáték, aminek az alapjai az 1800-as évekig vezethető vissza, ám ennek ellenére az első szabadalom mégis csak 1903-ban született meg a "háziúr" (The Landlord's Game) néven. Ez volt mindennek az alapja. Mára már olyan népszerűvé vált, hogy maga a Monopoly nevű társasjáték 37 nyelven érhető el, illetve 103 különböző országban értékesítik. Magyarországon is elég népszerű és a legtöbb család, ha nem is birtokol egyet otthon, legalább hallott már a játékról.

A játékmenet egyszerű. Két dobókockával dobunk, így bejárva az egész táblát, miközben telkeket és azokra házakat, vagy éppen hotelt veszünk, illetve cserélünk. Ha valaki rálép a másik tulajdonában lévő telekre, illetve ingatlanjára, az bérleti díjat fizet neki. A cél, egymást csődbe kell juttatni. A nyertes értelemszerűen az, aki a legvégén "talpon tud maradni".

A szabályzatot illetően van hivatalos, és léteznek egyedileg kreáltak is. Mivel a játék rendkívül sokoldalú lehet, így kompatibilis több szabállyal is. A játékmenet igen változatos lehet: vannak, akik a hivatalos szabályokat kedvelik, mások ezt megbonyolítják, leegyszerűsítik, vagy éppen teljesen átformálják.

1.2. Az ötlet

Az alapvető ötlet akkor született meg, amikor Monopoly szerű játékot kerestem, amit lehet multiplayerben játszani, viszont vagy nem volt ingyenes az adott játék, vagy csak lokális többjátékos módot támogatott vagy pedig valamilyen más egyéb szoftver kellett ahhoz, hogy játszani lehessen (pl. Steam).

Ekkor gondoltam ki, hogy csinálhatnék egy böngészős játékot, ami platformfüggetlen, ingyenesen lehet játszani és akár 2 kattintás elég ahhoz, hogy a barátokkal lehessen közösen játszani.

1.3. Hasonló játékok

Ahogy a fentiekben említettem, léteznek monopoly vagy monopoly szerű játékok megírva vagy böngészős játéknak vagy szoftvernek, de egyik sem felelt meg az általam meghatározott kritériumok minden pontjának. Például ott van a telefonos Monopoly, ami Androidra a Google Play Store-ban [6] vagy iOS-re App Store-ban [7] érhető el. Ezek fizetős játékok és csak telefonról érhetők el, nincs számítógépes megfelelője.

Ha az utóbbin szeretnénk ilyen jellegű játékkal játszani, akkor létezik egy úgynevezett Business Tour játék, ami Steam-en [8] található meg. Ez már ingyesen elérhető bárki számára, aki rendelkezik Steam fiókkal, illetve le is töltötte azt. Az opció, hogy barátokat meghívhassunk havi előfizetéshez kötött.

Nos, bizonyos oldalakon még találhatunk "Monopoly" játékokat, amiknek valószínűleg a Hasbrohoz semmi köze, ilyen például az Y8-on [9] található játék, ami úgyszint a Monopoly nevet viseli. Itt azt láthatjuk, hogy ingyenes, de csak lokálisan támogatja a többjátékos módot.

Az előbbivel egyáltalán nem összehasonlítható Monopoly játékot fejlesztett az Ubisoft, ami a Ubisoft oldalán [10] vásárolható meg. Rengeteg platformon elérhető, beleértve konzolok (Nintendo Switch / Xbox One / PlayStation 4), illetve Stadia-n [11], ami a Googlenek egy cloud gaming szolgáltatása.

Ha felépítésben hasonló játékokat nézzük, mint ami az elképzelt, akkor ilyennek tekinthető a skribbl.io [12], ami lényegében egy barkóba játék. Egy valaki mindig rajzol, a többieknek pedig azt kell kitalálni. Itt egy becenév (későbbiekben nickname) megadása után már indíthatjuk is a játékot. Kétféle lehetőség van a játékra.

Tudunk privát lobbyt létrehozni, amibe csak linken keresztül lehet belépni, illetve tudunk publikus szobákba csatlakozni, ilyenkor egy matchmakinget követően fog minket bedobni egy lobbyba. Ezt követően indul a játékmenet.

1.4. Elképzelés a játékról

A játék maga egy nagyobb projekt része, ami teljes mértékben saját ötlet és ennek csak egy része került megvalósításra a szakdolgozat keretein belül. Ezzel ellenben az elképzelés a teljes, végső játékról alkotott képet mutatja be. A játék megnyitását követően a kezdőlapra kerülünk, ahol, ha még nem játszottunk, automatikusan egy vendég (későbbeikben guest) felhasználót kapunk. Ez a guest felhasználó azt jelenti, hogy véletlenszerű szavakból alkotott felhasználónevet kapunk, így egészen vicces neveket kapva kezdhetjük meg a játékunk. Így is lehet majd játszani, de az eredményeink csak lokális eszközön tárolódnak el, így, ha másik eszközt szeretnénk a későbbiekben használni, ott már másik guest felhasználót kapunk. A főoldalon legyen lehetőségünk regisztrálni, illetve belépni. Ha guest fiókkal rendelkezünk, amivel már elértünk pontokat, akkor, ha az illető regisztrál, az elért teljesítmény onnantól kezdve hozzárendelődik a regisztrált fiókhoz és így bármely eszközön elérhetők lesznek azok a belépést követően. A főoldal fókuszában legyen egy input mező, ahol kódot tudunk megadni egy bizonyos lobbyhoz való csatlakozáshoz. Mellette legyen lehetőség egy új lobbyt kreálni, ami létrejöttekor úgyszintén kap egy kódot, amivel a későbbiekben be tudunk lépni. A lobby létrehozásakor változtathatók bizonyos paraméterek, amik akár befolyásolhatják a játékmenetet is. Erre példa, hogy kiválaszthassuk a játékmenet hosszát, ami lehet 15, 30, illetve 50 perc. A lobbyhoz nem csak kód és link segítségével lehessen csatlakozni, hanem a játékosok kilistázva is láthassák őket és csatlakozhassanak hozzájuk.

2. fejezet

Irodalmi áttekintés

A szakdolgozat során számos technológia, illetve szoftver vagy csomag lett használva. A Node.js [1] egy JavaScriptet futtató környezet, így az egész alkalmazás ebben lett megvalósítva. Az express [2] egy gyors, minimalista web keretrendszer Node.js-re, jelen esetben ez adja az alap szervert. A socket.io [3] egy alacsony késleltetésű kétirányú kommunikációt biztosít websocketen keresztül. A ReactJS [4] egy JavaScript könyvtár, ami felhasználói interfészek létrehozására találtak ki, kliensoldali felhasználásra. A PixiJS [5] egy HTML5 alkotói motor, aminek segítségével rugalmasan és könnyedén lehet nagyon szép digitális tartalmakat, akár teljesértékű játékokat létrehozni.

A Google Play - MONOPOLY - Classic Board Game [6], app Store - MONOPOLY - Classic Board Game [7], Business Tour [8], Y8 - MONOPOLY [9], Monopoly - Ubisoft [10] vagy Stadia [11] mind monopoly, vagy monopoly szerű játékok. A skribble.io [12] játék felépítésében hasonlít a monopoly-hoz.

A Balsamiq Wireframes [13], Figma [14], illetve diagrams.net [15] mind olyan szoftverek vagy oldalak, amivel látványterveket és diagramokat lehet készíteni.

A Material Icons [16] a Google egyik olyan könyvtára, ami ikonokat tartalmaz. A Mantine [17] egy React komponens könyvtár, amiben sok előre elkészített komponens található. Tailwind CSS [18] egy CSS keretrendszer, amivel gyorsabban és egyszerűbben lehet mondern appokat készíteni. Az i18next [19] egy olyan JavaScript keretrendszer, ami segítségével az alkalmazásokat többnyelvűvé lehet tenni.

A CC0 [20] egy licensz, ami a grafikai elemekre vonatkozik a játékon belül. A KenneY [21] és freepik [22] oldalról származnak a grafikai elemek.

Az npmtrends [23] oldalán bizonyos csomagokat tudunk összehasonlítani, hogy mit, hányszor töltöttek le. Ezt akár évekig visszamenőleg látni lehet. Ezáltal látható, hogy melyik csomagok vagy technológiák a legnépszerűbbek.

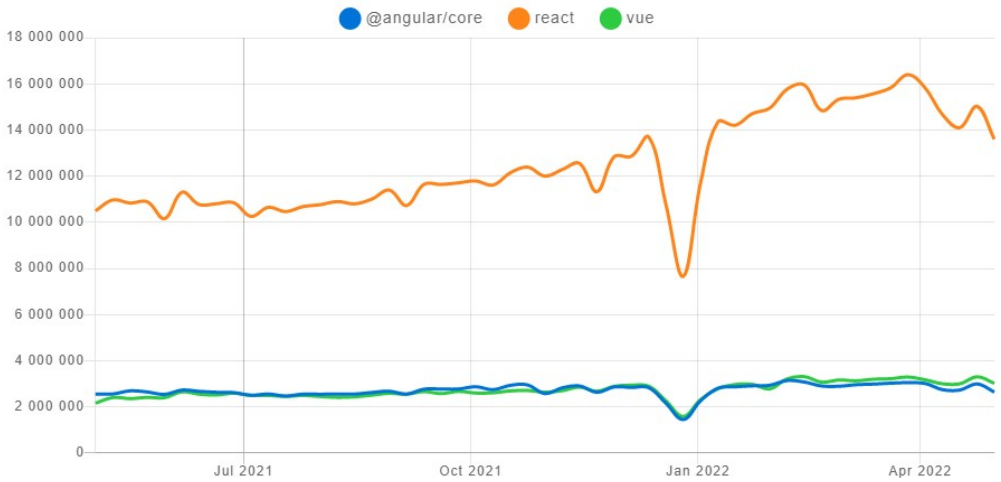
3. fejezet

Technológiai áttekintés

A fejezetben a nagyobb és főbb technológiákat ismerhetjük meg. Mik is azok valójában és mire használhatók. Illetve valamelynél szó esik arról is, hogy milyen cégek, illetve nagyvállalatok használják.

3.1. ReactJS

A ReactJS egy, a Facebook által fejlesztett könyvtár, amit azért hoztak kezdetben létre, hogy házon belül, könnyebben és gyorsabban tudják a felhasználói interfészeket létrehozni. Ez a mai napra akkorára nőtte ki magát, hogy a legnépszerűbb frontend technológiává vált és az elmúlt egy évben átlagosan több mint 12 millióan töltötték le hetente.



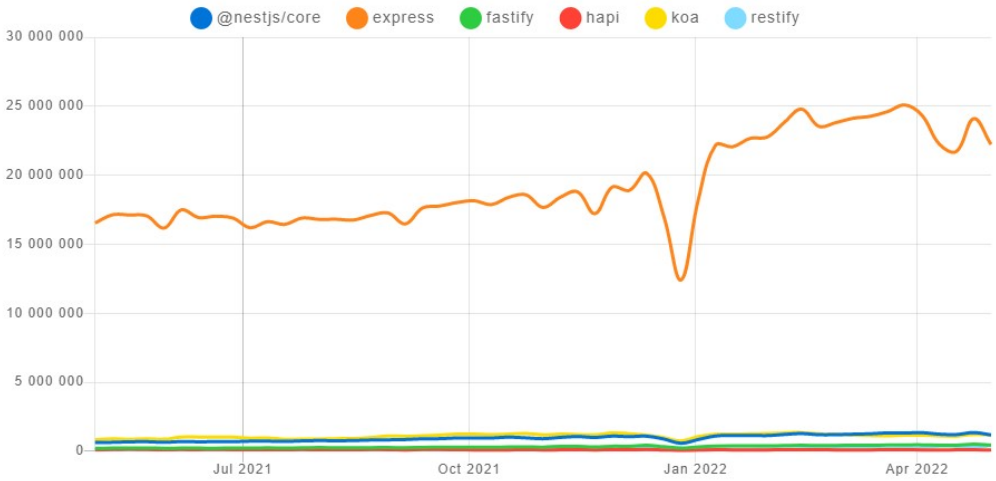
3.1. ábra. React, Angular és Vue letöltései az elmúlt évben az npmtrends [23] oldalán

Nagyon sokan azt állítják, hogy a React egy keretrendszer, pedig nem. Az el-

terjedtsége, illetve egyszerűsége miatt került erre a választás és persze a számtalan hozzá készült modulok és egyéb más könyvtárak miatt.

3.2. Express

Az express egy gyors, minimalista web keretrendszer. Leginkább szerverek létrehohására használják már lassan 11 éve. Ez idő alatt rengeteget fejlődött és mindenki szereti használni. Az utóbbit az bizonyítja, hogy hetente több mint 22 millió letöltéssel rendelkezik.



3.2. ábra. Express és hasonló keretrendszerek letöltései az elmúlt évben az npmtrends [23] oldalán

3.3. Socket.IO

Ez a technológia lehetővé teszi a websocketeken keresztül történő kétirányú kommunikációt alacsony válaszidővel. Mindezt úgy, hogy teljesen platformfüggetlen. Nagyon sokan használnak websocketet manapság, például chat applikációkhoz, illetve akár játékokhoz vagy bármiféle együttműködéshez, ahol többen szerkesztenek egy fájlt egyszerre. Például a Microsoft is websocketek segítségével éri azt el, hogy egyszerre többen tudjanak szerkeszteni egy dokumentumot. A játék során a teljes játékmenet és szobarendszer websocketeken kommunikál. Példaként lásd: 6.28. ábra.

3.4. PixiJS

A PixiJS egy 2D-s HTML5 alkotói motor, ami játékok készítésére is alkalmas. Rengeteg előre megírt modullal van tele, amik segítségével sokkal könnyebb és gyorsabb a fejlesztés. Számomra az előbb említetteken kívül azért volt jó választás, mivel rendkívül jól optimalizált, és ennek a technológiának a fejlesztőinek ez volt a fő cél. Pontosan ezért számos népszerű cég és vállalat használja ezt, olyanok, mint például az Adobe, Ubisoft, BBC, Disney, Google, HBO, Lego Marvel és még lehetne sorolni.

3.5. Material Icons

Ez egy nagyobb könyvtár része, az úgynevezett Material UI-é, röviden MUI. A MUI pedig a Material Design alá tartozik, amit a Google birtokol és fejlesztett ki. Ezek előre elkészített UI komponenseket tartalmaznak Androidra, iOS-re, webre és Flutterre. Ebben az esetben a MUI a webre készült, azon belül is a Reactra készült könyvtár, ami rengeget elemet tartalmaz. Többek között az ikonokat is, amiket a szakdolgozat keretein belül használtam.

3.6. Mantine UI

Reacthoz készült komponens könyvtárak közül talán ezt találtam számomra a legmegfelelőbbnek. Rengeteg előre elkészített komponens van, amiből például a modalt is használtam a játék készítése során. Ezek mellett sok hooks-t tartalmaz, mint például a useFullscreen Hook. Az utóbbit a játéknál használtam, hogy lehessen teljesképernyős módba állítani a játékot.

3.7. Tailwind CSS

A Tailwind CSS egy CSS keretrendszer, amit azért csináltak, hogy gyorsan és egyszerűen lehessen felhasználói interfészek kinézetét változtatni. Nagy mértékben testreszabható és rengetek előre megírt szükséges stílust tartalmaz. Azért választottam ennek a használatát, mivel szeretek egyedi dolgokat készíteni, és az előre megírt

komponensek nem annyira rugalmasak, mint nulláról megírni egy bizonyos elem kinézetét.

3.8. i18next

Az i18next egy olyan JavaScript keretrendszer, ami segítségével az alkalmazásokat többnyelvűvé lehet tenni. Számos keretrendszert és könyvtárat támogat és nem csak webes alkalmazás esetében használható. A készítőknek az volt a célja, hogy olyat készítsenek, amit egyszer kell megtanulni és utána bárhol lehet használni. Számos vállalat használja, például Firebase, mongoDB, Couchbase és még sok más.

4. fejezet

Feladat bemutatása

A fejezet bemutatja a feladat megvalósításának folyamatát. Mikor, milyen folyamat során mire került sor, illet milyen nehézségek akadtak a fejlesztés során.

4.1. Áttekintés

A szakdolgozat keretein belül megvalósult játék egy online társasjáték. Alapjául a monopoly nevű társasjáték szolgált. Az online társasjáték lényege, hogy egy szobarendszer segítségével alkotott 2-4 fő egymás ellen tudjon játszani. A lehető leghamarabb kell meggazdagodni, és a többi játékost csődbe juttatni. Mindezt úgy, hogy a játékban vannak hátráltatások és segítségek is.

A feladat megvalósítását azzal kezdtem, hogy felmérjem, hogy az ötletemhez mekkora igények társulhatnak és annak alapján tervezzem meg a rendszert. A rendszer felépítésénél törekedtem az egyszerűségre és a népszerű modulok használatára, többek között így került a választás a Reactra is.

Miután technológiailag megterveztem és felépítettem a rendszert, azt követően kezdődött el a szobarendszer megtervezése. Itt olyan dolgokat vettem alapul, hogy egyszerű legyen használni, egyértelmű, átlátható és gyors. Így jött az ötlet, hogy a szobákhoz tartozzanak kódok, ami alapján lehet csatlakozni. Ezek a kódok 6 számjegyűek, könnyen megjegyezhetők, így, ha egy adott személynek megadjuk, feltételezhetőleg meg tudja jegyezni. Ezek mellett fontosnak tartottam, hogy linken keresztül is meg lehessen hívni egymást, ezért a későbbiekben ezt is beleépítettem a rendszerbe. Ezek mellett úgy gondoltam, hogy legyen lehetőség a szobák megjelenítésére és azokhoz csatlakozni egy listából. Viszont a legfontosabbnak azt tartottam, hogy mindent valós időben lehessen látni. Itt gondolok arra, ha valaki nézi a szobákat, akkor közben lássa, ha létrejön egy új szoba, illetve csatlakoznak hozzá.

Ez azért kapott ekkora priorítást, mivel találkoztam már olyan megoldással, ahol nem lehetett a változásokat valós időben látni, hanem mindig rá kellett frissíteni az oldalra.

Ezt követően kezdődött a szobarendszer logikájának implementálása kliens, illetve szerver oldalon párhuzamosan, aminek során megtanultam használni a websocketeket. Ezután kezdődött a felhasználói felület szépítése. Ekkor merült fel az igény az egyéni színpaletta létrehozására. Lásd: 5.1. ábra.

Miután a szobarendszer implementálásával elkészültem, elkezdtem kigondolni a játék témáját. Azt tudtam, hogy monopoly szerű játékot szeretnék csinálni, de a témájával még nem voltam tisztában. Ekkor jutott eszembe az ötlet, hogy egyetem témájú játékot készítsek. Így gondoltam, hogy olyan városok lesznek a játékban, ahol híresebb vagy nevesebb egyetemek találhatók. Miután ezzel elkészültem, próbáltam valami szempont alapján csoportosítani őket egymáshoz, így csoportokat létrehozva. Az eredmény a következő lett:

Város	**Csoport színe**
Veszprém	barna
Budapest	barna
Debrecen	barna
Zürich	világoskék
Paris	világoskék
Edinburgh	világoskék
Chicago	pink
Austin	pink
Princeton	pink
Vienna	narancs
Berlin	narancs
Seattle	piros
New Haven	piros
London	sárga
Oxford	sárga
Cambridge UK	sárga
Berkeley	zöld
New York	zöld
Stanford	kék

Város	**Csoport színe**
Cambridge US	kék

4.1. táblázat. Városok és csoportjaik

Így ezekre a mezőkre egyetemeket lehet vásárolni és rálépés esetén tandíjat kell fizetni. Ezek mellett megszületett a múzeumok ötlete, majd végül a karantén, talár, erasmus és az adó is. Így gondoltam, hogy a maradék 3 mezőn esély kártyákat lehet majd húzni. Mivel ekkorra már megvolt a karantén ötlete, az, hogy PCR tesztet lehessen esély kártyából kapni, szinte egyértelmű volt. Ezt követően a mezők értékeinek megtervezése volt soron. Nagyon sok szempontot kellett figyelembe venni, hogy végül egy kiegyensúlyozott árrendszer legyen a játékban. Az első mező 1. szintje 200 000 játékbeli pénzbe kerül. A mezők 1. szintjét mindig az előző mező 1. szintjéhez adva 25 000-t kapjuk meg. A szintek 1-4-ig terjednek. A szintek közti ár úgy alakul, hogy az első szint 30%-ával emelkedik az összes szint. Így kalkulálódik ki a vételi ár. A tandíjat a vételi ár és a szintek függvényéből lehet kiszámolni. 1. szinten a vételár 55%-a, 2-on 70%, 3-on 85%, 4-en 100%-a. Ha egy mezőt már birtokol valaki, akkor azt a többi játékos a vételi ár duplájáért veheti meg. Amennyiben egy mezőt el kell adni, akkor a mező értékének a 90%-áért tehetik meg a játékosok. Amennyiben egy csoportban minden mezőt egy játékos birtokol, a tandíj 150%-ra emelkedik. Amennyiben duplázó kerül egy mezőre, akkor a bónusz nélküli összeg kerül duplázásra.

Így, hogy az alap elgondolás már megvolt a játékot illetően, a következő feladat a grafikai elemek keresése volt. Ezt követően komponensek készültek a különböző típusú mezőkből, majd a mezők logikája is implementálásra került. Szerencsére a játék fejlesztése során semmiféle nagyobb kihívással nem szembesültem, inkább csak az implementálandó logika és komponensek mennyisége volt nagyon sok, így az vett el rengeteg időt. Az állapota a játéknak egy nagy objektumban tárolódik, és a komponensek az objektum paramétereinek a változása hatására mozdulnak el, illetve végzik el a kívánt funkcionalitást. A játék fejlesztése során rengeteget tanultam és fejlődtem, mint technológia háttér szempontjából, mind ilyen komplex logika megvalósításában.

Miután a játék is implementálásra került, össze kellett kapcsolni a szobarendszert a játékkal. Itt kezdetben akadtak nehézségek, mivel a szobarendszer nem tárolt minden szükséges információt, ami kellett a játék elindulásához, viszont ez ki lett egészítve és onnantól már könnyedén ment minden.

5. fejezet

Tervezés és modellezés

A tervezési fázisban fontosnak tartottam, hogy a lehető legpontosabban megtervezzem magát a webapplikáció kinézetét, felépítését és működését. Így a színektől kezdve az architektúrán át, egészen a látványtervekig, minden, a tervezéshez és a modellezéshez kapcsolódó fontos dologról olvashatunk a fejezetben.

5.1. Színek

A tervezés legelső lépéseként a színek, árnyalatok kiválasztása volt a legfontosabb. Szerettem volna egy olyan színösszeállítást kitalálni, ami nem annyira világos, kellemes a szemnek, illetve a kontrasztarány megfelelő legyen. Így készült el a következő színpaletta:



5.1. ábra. Egyedileg összeállított színpaletta

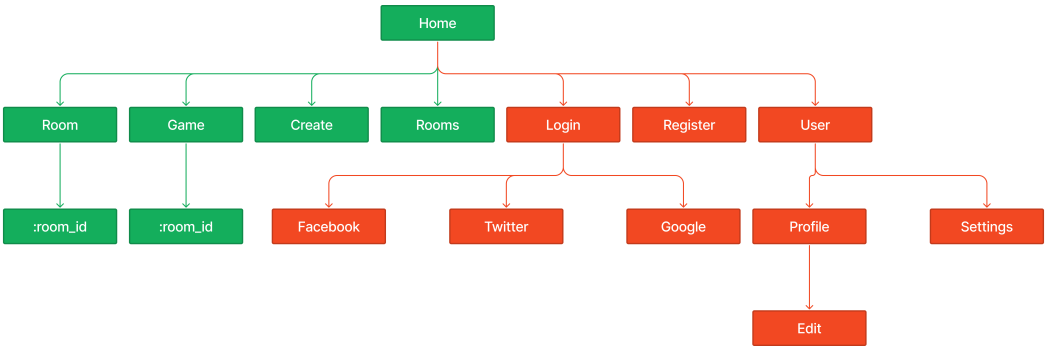
Ezt a 9 színt találtam megfelelőnek, amik nem annyira erősek és a legsötétebb és legvilágosabb szín között sem bántó a kontraszt, de mégis jól látható egymáson. Ezt azért tartottam szükségesnek, mivel a mai világban egyre többet nézzük a

monitorokat, kijelzőket és a túl nagy kontraszt bántja és hamarabb fárasztja a szemet, ami egészségtelen. A legtöbb populáris oldal, ahol több időt töltenek az emberek nap mint nap, a sötét mód megléte már szinte alapnak tekinthető, mind a nagyvállalatok részéről is, akik kímélni próbálják a felhasználók szemeit, továbbá a felhasználók részéről is elvárás a sötét mód megléte.

5.2. Oldaltérkép

Mint azt már korábban is említettem, a projektnek nem minden része a szakdolgozat keretein belül valósult meg, így az oldaltérkép sem a jelenlegi állapotot mutatja. Pontosan ezért kiemelésre került zöld színnel, ami már kész, illetve piros színnel, ami majd csak a továbbiakban, szakdolgozat keretein kívül kerül implementálásra. Az utóbbiakról olvashatnak a "További lehetőségek/továbbfejlesztési tervek" című fejezetben a 67. oldalon.

Az oldal felépítése a következőképpen néz ki:



5.2. ábra. Oldaltérkép

Felépítése az alábbi:

- Home: ez a főoldal, ide érkezünk amikor először megnyitjuk a webapplikációt és a későbbiekben innen érjük el az alpontokat.
  - Room: a szobát jelenti, itt láthatjuk a szobát, amiben vagyunk, illetve az adatait.
    - :room_id: a szoba kódját jelzi, amiben éppen tartózkodunk.

- Game: a megadott szoba játékmenetét mutatja.
  - :room_id: a szoba kódját jelzi, amiben éppen tartózkodunk.
- Create: itt hozhatunk létre új szobát.
- Rooms: ezen az oldalon láthatjuk a meglévő szobákat, illetve csatlakozhatunk hozzájuk.
- Login: authentikációs oldal.
  - Facebook: facebook bejelentkezés érhető el.
  - Twitter: twitter bejelentkezés érhető el.
  - Google: google bejelentkezés érhető el.
- Register: ezen az oldalon tudunk új fiókot létrehozni.
- User: itt láthatjuk a felhasználónk adatait, ha beléptünk.
  - Profile: profilunk adatai jelennek meg.
    - Edit: az előbbieket szerkeszthetjük.
  - Settings: beállításaink találhatók itt.

5.3. Architektúra

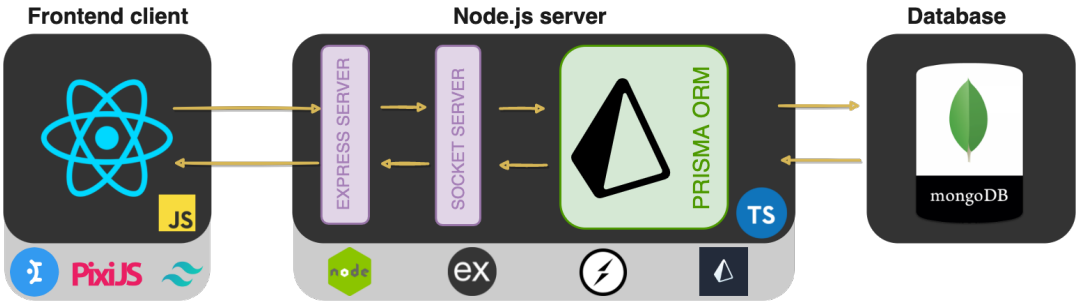
Kliens oldalon az alkalmazás React segítségével került implementálásra, ami egy Facebook által fejlesztett „könyvtár". Így a weboldal komponensekből épül fel és szükség esetén nem az egész oldal frissül, csak az a komponens vagy elem, amin a változás történik. Ezzel optimalizáltabb és gyorsabb oldalt tudunk létrehozni, és nem mellesleg a fejlesztést is gyorsítja, megkönnyíti. Szerver oldalon Node.js szerver található, továbbá van egy Express és egy Socket.io szerver is.

Az Express szerver felel a kérések kiszolgálásáért, illetve a middlewarekért is. A middleware az egy olyan réteg, ami a kérések és a hozzájuk tartozó logika között van. Például hitelesítésre használjuk. Ha beérkezik egy kérés, akkor hitelesíti a felhasználót, hogy jogosult-e a kérés meghívására, majd, ha megfelelőnek találja, akkor szolgálja ki a felhasználót.

A socket szerver pedig az események fogadását és küldését biztosítja.

Adattárolást illetően mongoDB-t használok, ami egy NoSQL adatbázis, így nagy adattömegek között is gyors a keresés, illetve, azokat könnyű módosítani. ORM-ként Prisma került alkalmazásra. Az ORM nagyban segíti a programozókat a különféle adatbázisok elérésében. Több típusú adatbázist is elérhetünk ugyanazzal a

konzisztens kóddal és logikával. Ennek az egyedüli hátránya a bizonyos adatbázis specifikus változók, funkciók hiánya. A Prisma segíti a fejlesztőket a gyorsabb és hibamentesebb kód elkészítésében. Az alábbi képen látható az architektúra:



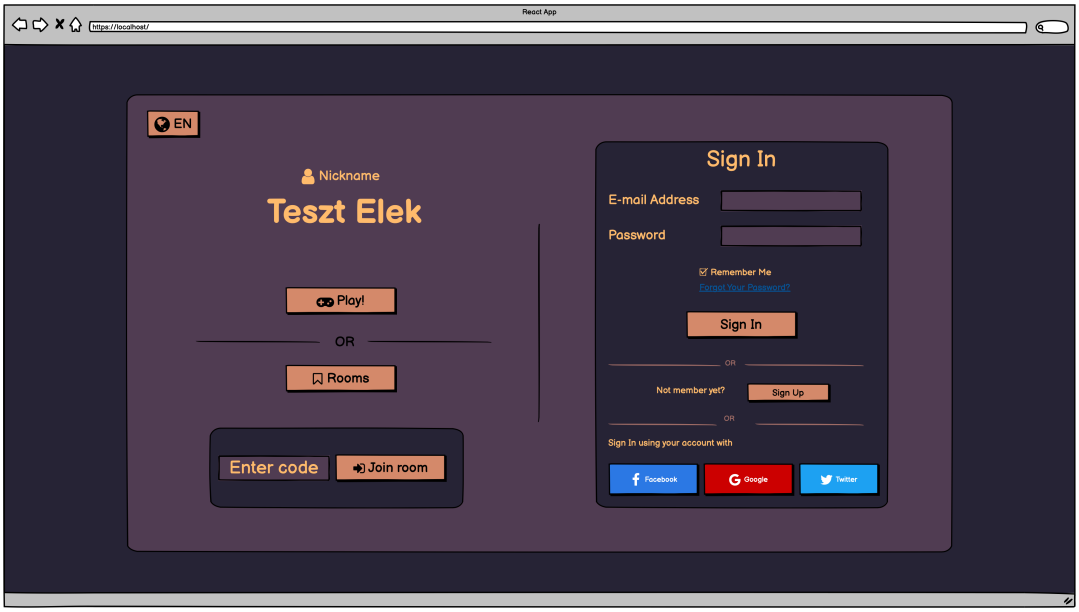
5.3. ábra. Architektúra

A kliens websocketen keresztül kommunikál a szerverrel. Itt az adatcsomagok eseményeken keresztül kerülnek továbbításara annak függvényében, hogy melyik félnek van szüksége adatra. Amennyiben a kérés megköveteli bizonyos dolgok kiszámítását, az szerver oldalon hajtódik végre. Az adatbázist pedig az ORM segítségével éri el a szerver. Az adatbázishoz közvetlenül nem fér hozzá a kliens.

5.4. Látványtervek

A látványtervek készítése során a már korábbi színek kerültek felhasználásra. Ezek során három programot/webappot is használtam. Az első a Balsamiq Wireframes [13] volt, amiben az oldalakról készültek a látványtervek. A következő a Figma [14] volt, amit személy szerint nagyon kedvelek, mert letisztult és könnyű használni. Az utóbbival a(z) Egyedileg összeállított színpaletta és a(z) Oldaltérkép képeket készítettem el. Majd végül a diagrams.net [15] segítségével a(z) Architektúra kép került elkészítésre.

5.4.1. Főoldal

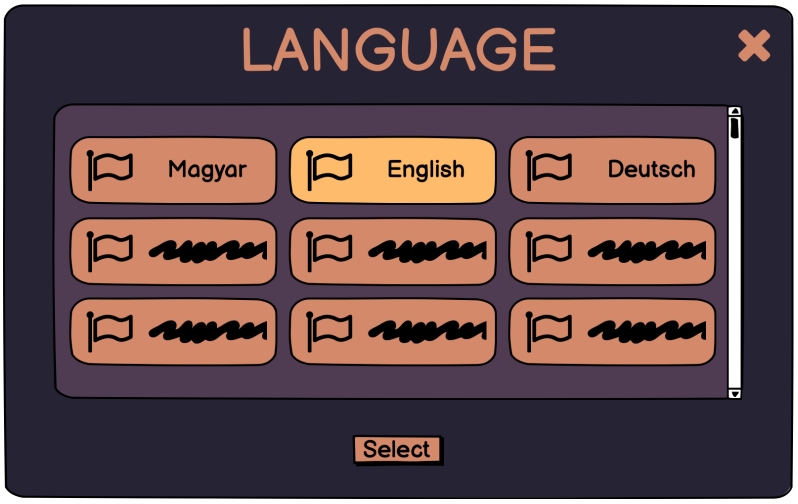


5.4. ábra. A főoldal látványterve

Ezt látjuk amikor megnyitjuk a web appot. Jól láthatóan ketté van osztva a főoldal. A bal fele a játékhoz és a szobarendszerhez kapcsolódik, a jobb oldal viszont a felhasználó authentikálásához.

Az utóbbinál 4 lehetőség van a bejelentkezésre, email címmel, Facebookkal, Google fiókkal, illetve Twitterrel. Amennyiben a játékfülön a *Play!* gombra kattintunk, akkor elindul egy matchmaking és az oldal kreál nekünk egy szobát, majd, ha megtelik automatikusan bedob a játékba. A *Rooms* gombra kattintva megjelenik a /rooms oldal, ahol az összes létező szobát láthatjuk, illetve csatlakozhatunk is valamelyikhez. Ám amennyiben a barátunk elárulja a szoba kódját, akkor csak a kódot kell beírni az *"Enter code"* mezőbe, majd a *Join room* gombra kattintva beléphetünk a szobába. Illetve ezeken felül látható a bal felső sarokban egy nyelvválasztó menü is.

5.4.2. Nyelvválasztó menü



5.5. ábra. A nyelvválasztó menü látványterve

Itt kiválasztható, hogy milyen nyelven szeretnénk játszani.

5.4.3. Szobák



5.6. ábra. A szobák látványterve

Ezen a lapon láthatjuk az összes létező szobát. A szobáknál látni lehet a szoba nevét, azt, hogy mennyi ideig tart a játék, illetve, hogy mekkora férőhelyes és jelenleg hányan, illetve kik tartózkodnak a szobában. Ezen felül a jobb szélén látható gombbal lehet csatlakozni a szobába. A jobb felső sarokban található gombbal tudunk új szobát létrehozni.

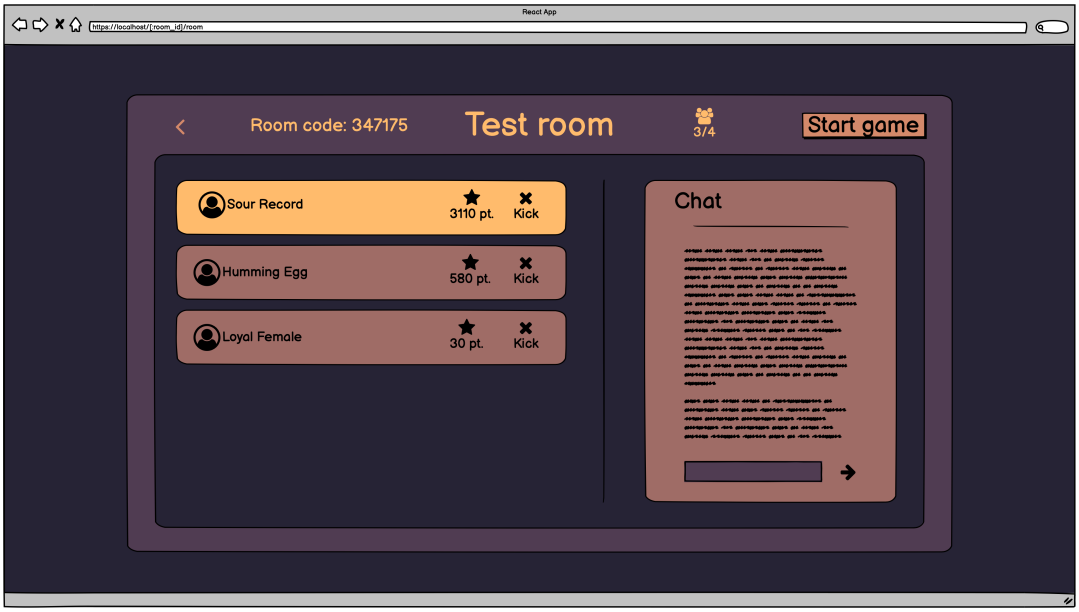
5.4.4. Új szoba létrehozása



5.7. ábra. Szoba létrehozásának látványterve

Itt meg tudjuk adni, hogy mi legyen a szoba neve, illetve az időt, majd, hogy mennyien tervezünk játszani. Az utóbbi kettőt a két legördülő listából tudjuk kiválasztani. Ezen felül, ha privát szobát szeretnénk készíteni, van lehetőségünk jelszót megadni, ellenkező esetben mindenki csatlakozhat majd a játékunkhoz.

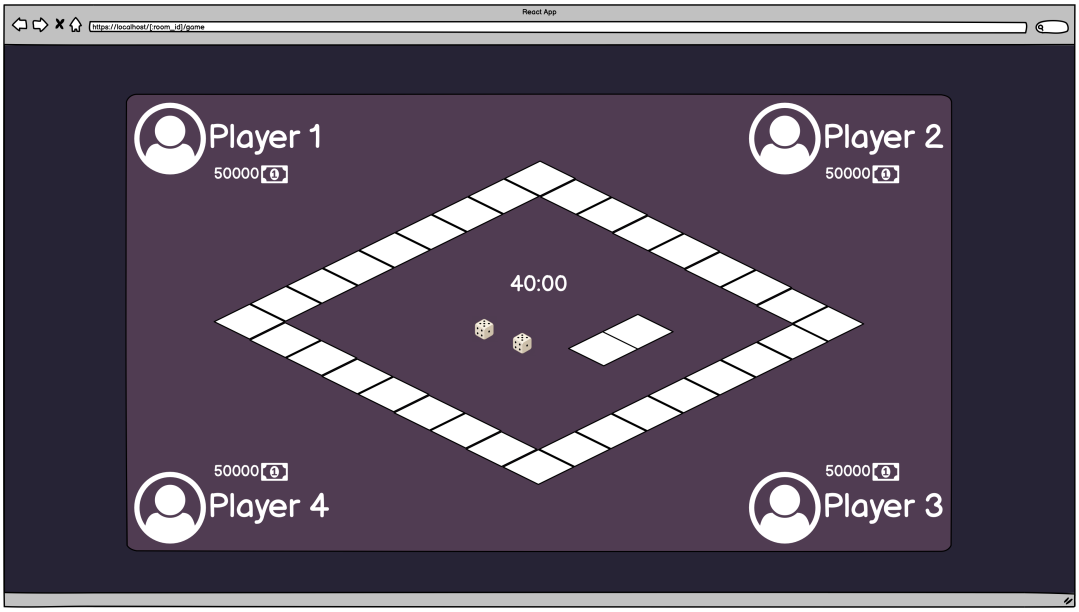
5.4.5. Szoba



5.8. ábra. Szoba látványterve

Itt láthatjuk, hogy néz ki amikor még nem indult el a játék, de a játékosok a szobában várakoznak az érkező játékosokra. Látni egymás pontszámát, nevét, képét, illetve, ha szeretnének, akkor rendelkezésükre áll egy chatszoba is. Aki elkészítette a szobát az rendelkezik kick joggal, így a nem kívánatos személyeket ki tudja rúgni a szobából. Amennyiben szeretnénk másokkal megosztani a szobánk kódját, akkor a szoba neve mellett balra az is látható, illetve rákattintva egyből vágólapra kerül. Amennyiben, ha megvan a létszám, akkor a jobb felső sarokban lévő gombbal bármikor el lehet indítani a játékot

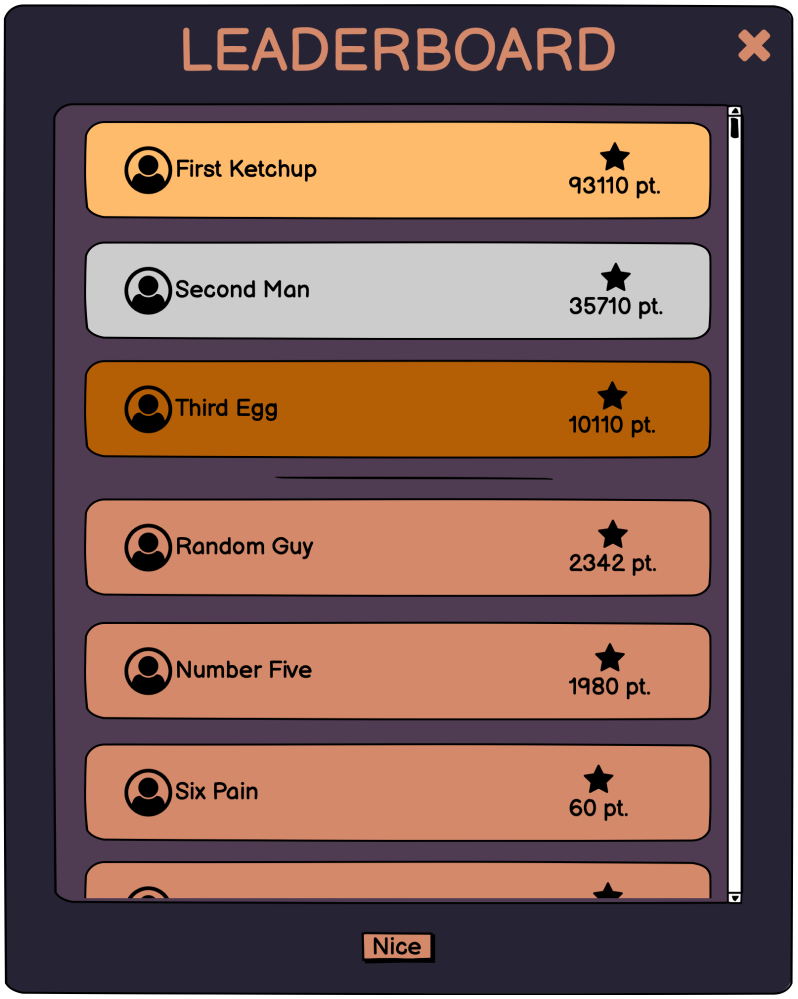
5.4.6. Játék



5.9. ábra. Játék látványterve

A játék mezői izometrikus nézetben kerülnek elrendezésre, így átláthatóbb, mintha nem abban lenne, és még jobban is néz ki. Ez ad neki egy kis 3D-s hatást, miközben továbbra is csak 2D-s elemek vannak izometrikus nézetbe elhelyezve. A játéktábla közepén található a visszaszámláló, ahol láthatjuk mennyi időnk van hátra a játék vége előtt. Ha ez lenullázódik a játék véget ér és kihirdeti a nyertest. A nyertes az lesz, aki ez idő alatt a legtöbb vagyonra tett szert. A játéktáblán találhatók még a szerencse és meglepetés kártyák, illetve a 2 dobókocka. A dobókockák gombnyomásra dobódnak el és 2 db véletlenszerű számot adnak vissza egytől hatig. A két dobott érték összeadódik, majd a játékos az így kapott számnyi mezőt lép előre az óramutató járásának megfelelő irányban. A játékosokat az óramutató járásával megegyezően a bal felső sarokból indulva láthatjuk a képernyőnk sarkaiban. A pénz a nevek alatt látható. A birtokolt vagyont sehol nem írja, így kicsit izgalmasabb a játék, mivel sok helyzetben nem feltétlen lesz egyértelmű, hogy ki hányadik jelenleg, így ez csak a játék végén derül ki. A játékosoknak a képei is látszódni fognak, de ezzel ellentétben a chat nem. Így a játékosok maximálisan a játékra tudnak figyelni és nem húzzák az időt feleslegesen.

5.4.7. Ranglista



5.10. ábra. A ranglista látványterve

A ranglistán a játékosok a szerzett pontjaik alapján szerepelnek csökkenő sorrendben. A ranglista minden héten nullázódni fog, hogy minden játékos kapjon esélyt a helyezésre. A lenullázással csak a ranglista nullázódik, a pontok továbbra is megmaradnak a profilban. Akik az első három helyezettek lesznek, azok a következő héten egy kitűzőt kapnak nevük mellé és büszkén viselhetik.

6. fejezet

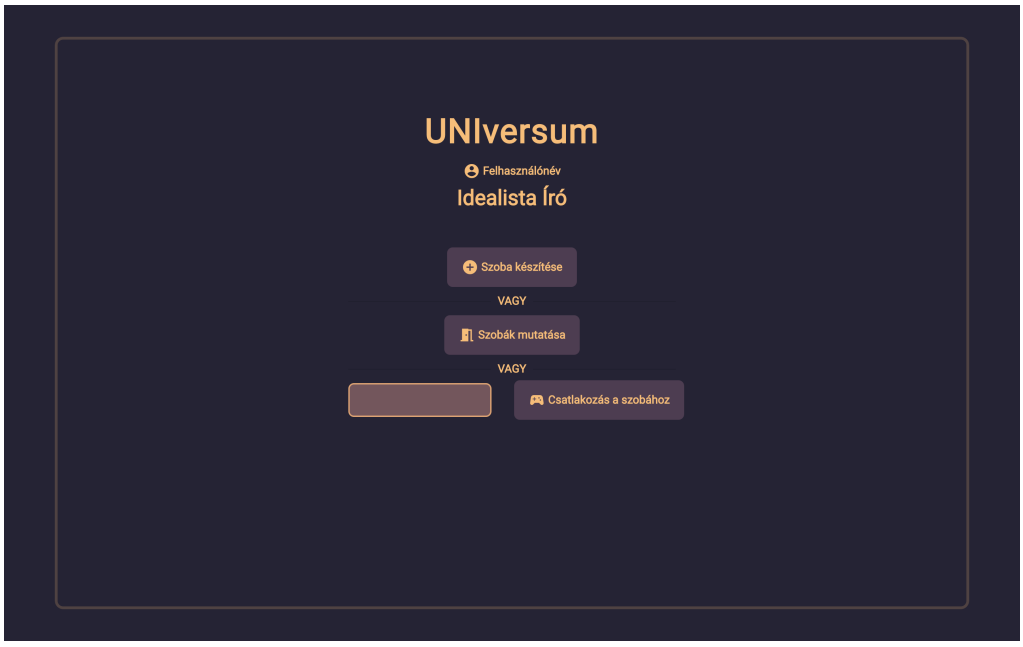
Implementáció és tesztek

6.1. Implementáció

Az implementációt követően már láthatjuk a látványtervekhez képest mikben is van különbség. Ez a fejezet arról szól, hogy mi került implementálásra, illetve, hogyan. Ez kettő részre bontható, az egyik a szobák menedzselésével kapcsolatos, a másik pedig maga a játék, illetve játékmenettel.

6.1.1. Szobarendszer

6.1.1.1. Főoldal



6.1. ábra. A főoldal

A főoldalon megtalálunk mindent, ami a szobarendszer működéséhez szükséges. Amikor megnyitjuk az oldalt, akkor kapunk egy automatikusan generált nevet, amit kettő óriási tömbből választ ki az algoritmus véletlenszerűen ismétlődés nélkül. Ezek kliens oldalon találhatók meg és a generálás után ezt a *set_username* nevű eventen, websocketen keresztül küldi el a szervernek. Ez minden újratöltésnél újra és újra legenerálódik, így egy személy nem tudja megtartani a kapott nevet. Minden egyes alkalommal amikor egy felhasználó csatlakozik a websocket szerverhez, eltárolódik egy Map-ben, aminek a kulcs értéke a csatlakozáskor létrejött socket id, az értéke pedig a felhasználó szobáját tartalmazza, amennyiben van, ezen felül pedig az előbb említett username-t, azaz a felhasználónevet.

Az oldalon 3 gomb található, attól függően, hogy a játékos milyen interakciót szeretne, annak megfelelően lép tovább az oldal. Amennyiben a *Szoba készítése* gombra kattintunk, átirányítódik az oldal a /create oldalra. Ekkor tudunk létrehozni új szobát. Amennyiben a *Szobák mutatása* gombra megyünk, akkor az oldal átirányítódik a /rooms oldalra. Végül, ha rendelkezünk egy szobának a kódjával, akkor azt az alsó mezőbe illesztve tudunk csatlakozni ahhoz. Ekkor már a /room/:room_id oldalon vagyunk, ahol a :room_id a szoba kódja. Az oldalon megtalálható kis ikonok a Material icons [16] csomagban megtalálható ikonok közül származnak.

6.1.1.2. Szobák



6.2. ábra. Szobák listázása

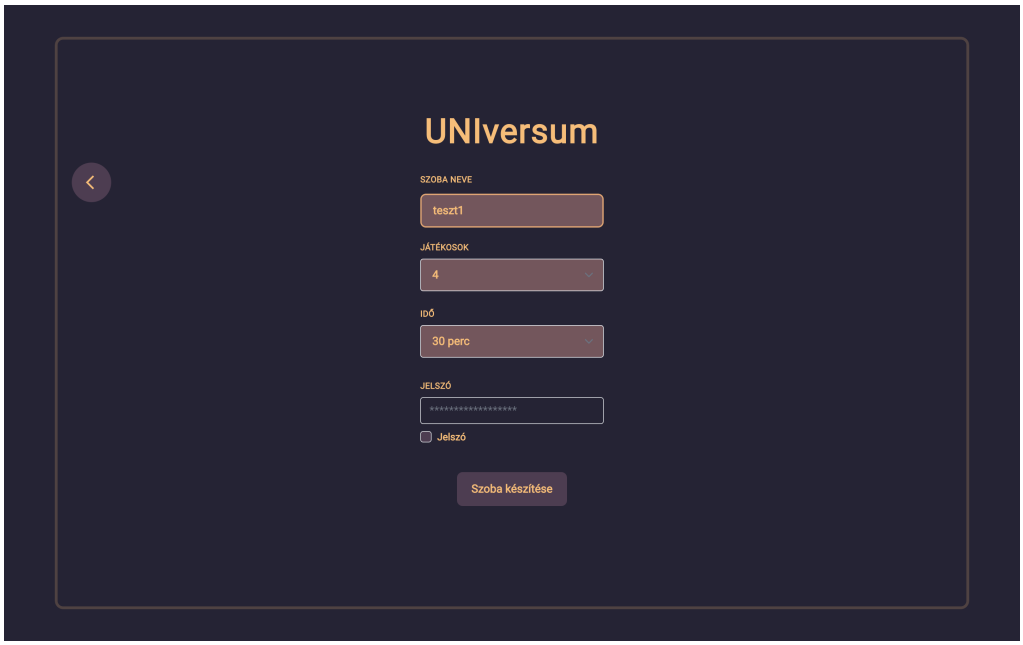
Amennyiben a *Szobák mutatása* gombra rámentünk vagy a /rooms URL-t nyitottuk meg, abban az esetben láthatjuk a jelenleg létrehozott szobákat, annak paramétereit és a szobában tartózkodók nevét. Ezt az oldal betöltésekor kéri le a szervertől a *get_rooms_req* eventtel, illetve a *get_rooms_res* -re feliratkozva kapja meg. A szobák és annak adatai a szerveren tárolódnak el egy Map-ben. Az utóbbiban a szoba kódja a kulcs és az értéke a szoba adatait tartalmazza. A szoba adatai alatt a benne tartózkodó felhasználók adatait, a szoba kódját, a nevét, férőhelyét, jelszavát hashelve amennyiben van, privát vagy publikus, illetve a játék menetidejét értjük.

A listában az utóbbiakból csak párat látunk, ilyen a szoba neve, típusa, férőhelye. Kiválasztva a szobát, jobb oldalt jelenik meg a benne tartózkodók nevei. Csatlakozási szándékunkat a *Join* gombra rányomva tudjuk érvényesíteni. Amennyiben publikus a szoba, egyből beléptet minket a szobába, ha nem, jelszót kér, és csak a jó jelszó esetében nyerünk belépést az adott szobába. A belépés nem lehetséges abban az esetben, ha a szoba tele van. Ekkor a gombra rá sem tudunk kattintani.

Belépéskor a rendszer átirányít minket a szoba URL-jére, ahol többek között websocketen keresztül történik a szoba adatainak újbóli lekérdezése és olyan eventekre való feliratkozás, aminek köszönhetően valós időben láthatjuk a szobákat érő

változást. Ilyen például az új felhasználó belépése, vagy kilépése.

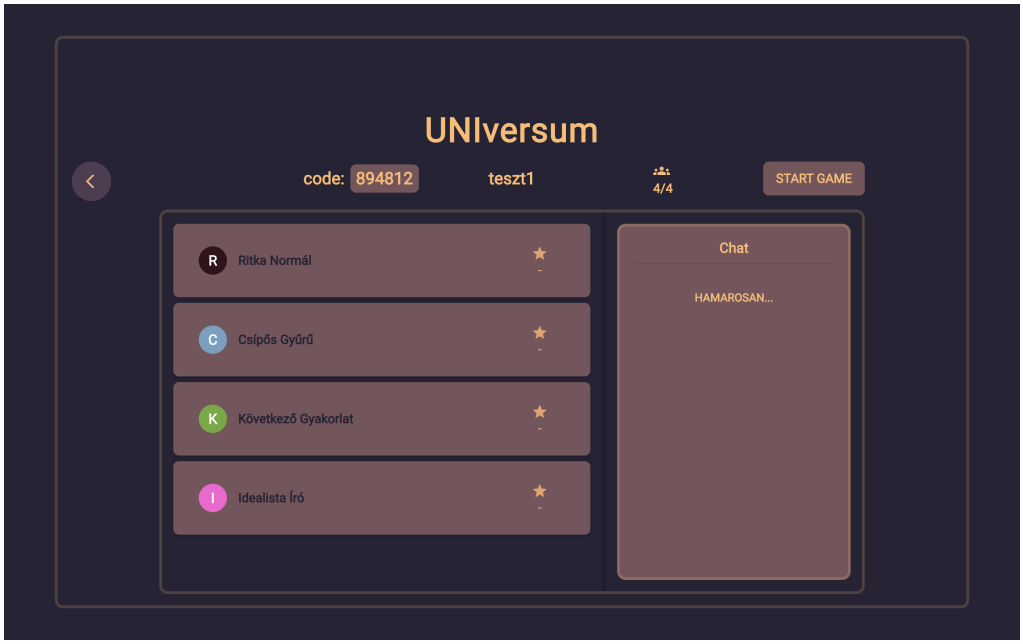
6.1.1.3. Új szoba létrehozása



6.3. ábra. Új szoba létrehozása

Itt egy egyszerű űrlapot láthatunk, aminek kitöltésével új szobát hozhatunk létre. Az összes paraméterét itt adhatjuk meg a szobának. Ezek nem mások, mint a szoba neve, játékosok száma, játékmenet hossza, típusa, és privát szoba esetén jelszava. Amennyiben rákattintunk a *Szoba készítése* gombra, a szoba adatai, illetve a létrehozó felhasználónak az adatai kerülnek elküldésre a szervernek, ahol egy új 6 számjegyű kód kerül legenerálásra és ezt követően létrejön a szoba, majd a létrehozó felhasználó automatikusan beléptetésre kerül.

6.1.1.4. Szoba



6.4. ábra. Szoba

Ez a látvány fogad minket a szobába való belépést követően. A felső részen látható a szoba kódja, amire rákattintva, az vágólapra kerül. Emellett láthatjuk a szoba nevét, férőhelyét, illetve a játék megkezdéséhez szükséges gombot. Az utóbbi csak akkor elérhető, ha a szoba megtelt.

Bal oldalon a listában láthatjuk a játékosok neveit, illetve a játékos pontszámát, ami jelenleg a felhasználói fiókok authentikálásának hiányában nincs összekötve valós adattal.

A játékosok kidobása a szoftvernek ebben a verziójában még nem elérhető. A chat a látványtervnek megfelelően jobb oldalra került, viszont ez a funkció a szakdolgozat keretein belül nem került implementálásra.

6.1.1.5. Nyelvválasztó menü



6.5. ábra. Nyelvválasztó menü

A szakdolgozat keretein belül ez is implementálásra került. Ennek megvalósítására i18next[19]-et használtam. Eredetileg a játékon kívül mindenhol ott lenne a lehetőség a bal felső sarokban a nyelv kiválasztására, de ez az opció ideiglenesen ki lett kapcsolva, mivel a szakdolgozat keretein belül nem lett lefordítva 100%-ra egyik nyelv sem, így viszont nem éreztem relevánsnak a meglétét. Ezért jelenleg az oldal csak magyar nyelven érhető el.

6.1.1.6. Websocket események

Az alábbi táblázat a szobákhoz tartozó websocket eseményeket tartalmazza, illetve azok rövid leírását. Ezek mellett megtalálható benne, hogy kliens vagy szerver oldalról történik-e az esemény küldése. Értelemszerűen, ha valami kliens oldalról van elküldve, akkor ugyanaz szerver oldalon fel van iratkozva arra az eseményre, azaz, lehallgatja azt az eseményt. Fordított esetben a szerver küldi és a kliens lehallgatja ugyanazt az eventet.

Event név	Leírás	Kliens/szerver
set_username	Elküldi a felhasználónevet, szerveren beállítódik.	kliens
create_room	Szoba adatait elküldi, szerver létrehozza.	kliens
get_room	Létrehozott szoba adatait elküldi a kliensnek.	szerver
join_room	Csatlakozási kérés egy szobába.	kliens
joined_room	Csatlakozási kérelemről való visszajelzés. (pl.: sikerült e)	szerver
user_joined	Szobában lévők megkapják az újonnan csatlakozott adatait.	szerver
leave_room	Kilépési kérelem indítása.	kliens
leaved_room	Kilépési kérelemről való visszajelzés	szerver
get_rooms_req	Szobák adatainak lekérése.	kliens
get_rooms_res	Szobák adatainak küldése kliensnek, amennyiben változtak.	szerver
get_room_by_id_req	Szoba lekérése a kódja alapján.	kliens
get_room_by_id_res	Szoba adatai elküldése kliensnek.	szerver
start_game_req	Játék indításának kérelmezése.	kliens
start_game_res	Játék elindítása, felhasználók átirányítódnak a játékba.	szerver

6.1. táblázat. Szobarendszerhez tartozó eventek

6.1.2. Játék

Ez az alfejezet mindent tartalmaz a játék implementálásáról, a játék mögötti logikáról, illetve a felhasznált elemekről.

6.1.2.1. Játék felépítése

Alapvetően a játék kettő, viszonylag nagy objektumból áll. Az első felel a játék legenerálásáért, hogy melyik mező milyen típusú, mi a neve, illetve, hogy melyik csoporthoz tartozik. Ez tárolja még az esély kártyákat is, hogy mi a nevük, leírásuk, illetve, milyen bónuszt ad, milyen mennyiségben. A második objektum pedig a játék aktuális állapotát írja le.

Az utóbbi olyan fontos paramétereket tárol, mint például a jelenlegi játékos, aki lép és interakciót végez. Vagy például ez tárolja, hogy lehet-e dobni, vagy éppen vásárolni új egyetemeket. Továbbá egyéb más fontos információkat is számontart, ide köthető az eladás logikája is, mivel eltárolódik, hogy milyen értékben szüksé-

ges eladnia az adott játékosnak és itt határozódik meg, hogy mit szán eladásra. Maga az eladás is egy reducer meghívásával történik, ami a paramétereket állítja át. A programban a különböző komponensek ezeknek a paramétereknek a változását figyelik és aszerint generálódik le mindig az aktuális interakció következménye. Ezeken kívül, olyan dolgok, mint a mezők kiválasztása is az objektum paramétereinek változtatásával történik. Külön paraméterek segítségével olyanokat tudunk beállítani, hogy egy vagy több mező kiválasztása lehetséges, illetve, hogy csak a saját tulajdon vagy a semleges területek is kiválaszthatók-e. Az utóbbira példa az Erasmus. Ezek mellett nem csak logikát tárolnak, hanem olyan dolgokat is, mint például a körönkénti bónusz értékét, vagy a múzeumok árát.

Maga az egész karantén rendszernek a megvalósítása is ezekkel történik. Eltárolja, hogy ki van karanténban és hány körből marad még ki. Illetve, ha az illetőnek van negatív PCR tesztje, az is itt tárolódik el.

Itt láthatunk egy példát az utóbbiról, illetve a mezők tulajdonságairól.

- players: a játékosok (jelen esetben csak egyet mutatok be)
  - username: felhasználónév
  - userId: felhasználó egyedi azonosítója
  - colorCode: színkódja a játékosnak
  - money: pénze a játékosnak
  - hasPCR: van e PCR tesztje
  - coutPCR: mennyi PCR tesztje van
  - isInQuarantine: karanténban van-e az illető
  - QuarantineRounds: hány kör van még hátra a karanténból
  - field: milyen mezőn áll
  - timeRemain: mennyi ideje maradt az interakciót végrehajtani
  - playerCountdown: ő lép e jelenleg
  - isBankrupt: csődbe ment e
- map: a mezők tulajdonságai (jelen esetben csak egyet mutatok be)
  - id: mező azonosítója
  - level: rajta lévő tulajdon szintje

  - ownerColor: milyen színű játékos birtokolja
  - hasDoubler: van e rajta talár
  - doublerCount: hány talár van rajta

A players paraméter valójában egy tömb, ami annyi játékost tárol, ahány éppen játszik a jelenlegi játékban. A map szintén egy tömb, ami a 32 pályának a tulajdonságát tárolja le.

Az előbbiek a játék folyamán kliens oldalon tárolódnak Redux store-ban, és reducerek meghívásával változnak a paraméterek állapotai. A reducerek azok, amik bizonyos állapotot tudnak változtatni, illetve lehet átadni nekik paramétereket. Hasonló elven működnek, mint az eljárások, csak reducer nem hívhat meg reducert. Így lényegében reducerek vannak írva a játékhoz, egészen pontosan 36 reducer, és ezek hívódnak meg a játék folyamán, annak függvényében, hogy ki hova lép, milyen fizetési kötelezettsége adódik, illetve milyen interakciót végez.

6.1.2.2. Mezők generálása

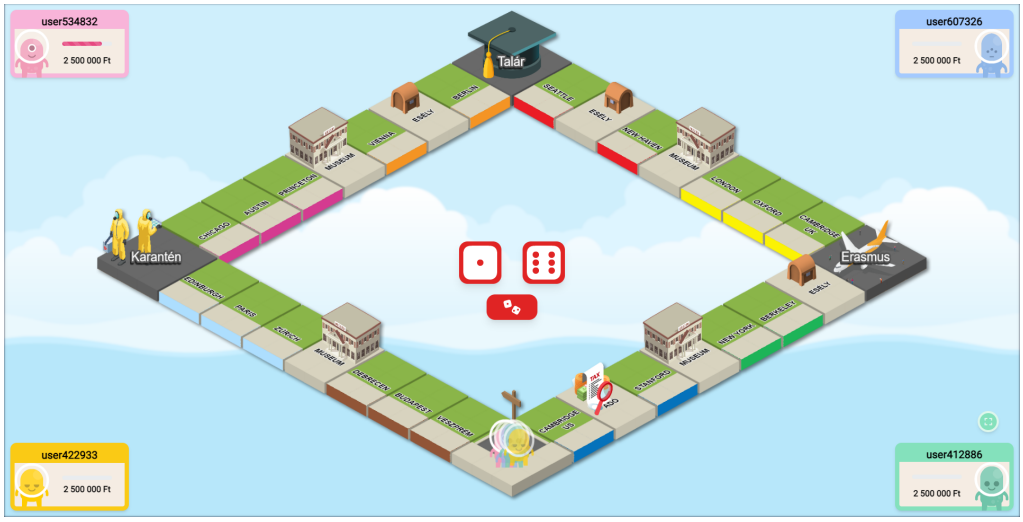
A mezők generálása azzal kezdődik, hogy a szoftver végigiterál azon az objektumon, ami letárolja a mezők típusait, tulajdonságait, majd a kódjukra meghívja azt a függvényt, ami kiszámítja a koordinátájukat a 2D-s térben. Ennek a függvénynek elég paraméterként megadni az adott mező kódját és az elvégzett számításokat követően visszatér az X és Y koordinátával. Az utóbbi függvény nem csak a mezők, hanem a játékosok helyzetét és számolja. Ezután a program megnézi az adott mezőnek a típusát (pl.: normál, sarok elem, múzeum, stb.) és olyan komponenst rederel ki arra a koordinátára. Ezek a komponensek az alábbiak lehetnek:

- Sarok komponenst
  - Karantén
  - Erasmus
  - Talár (duplázó)
  - Start mező
- Normális komponenst
- Múzeum komponenst
- Esély komponenst

- Adó komponenst

Az utóbbiak ezek után kerülnek részletes kifejtésre.

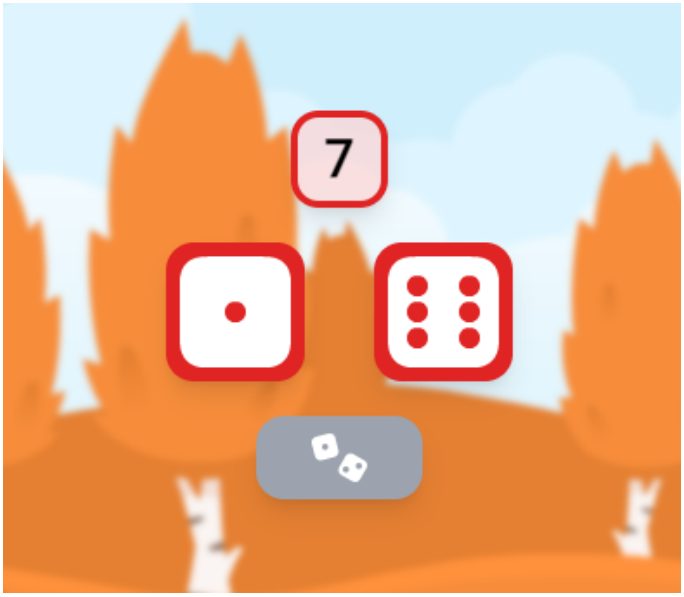
Íme egy kép, ami a generálás utáni kezdő állapotot mutatja:



6.6. ábra. Kezdő állapot a mezők generálása után

6.1.2.3. Dobókockák

A dobókockák azért szükségesek, hogy a játékosok dobni tudjanak 1-től 12-ig terjedő számot, így megkapják azt, hogy hány mezőt kell lépniük. 2 dobókocka van, mindig olyan színű kerettel és háttérrel, mint, amilyen színű játékos éppen soron következik. Eldobásukkal kettő véletlenszerű szám generálódik 1 és 6 között, majd ezután ezek összeadódnak és beállítódik az utolsó dobás értéke ennek az értékére.



6.7. ábra. Dobókocka dobás után

Dobásnál ellenőrzésre kerül, hogy az illető karanténból dob-e, mivel ilyenkor a játékos csak akkor mozdulhat el, ha kiszabadul a karanténból, amihez viszont 12-t kell dobnia. Amennyiben nem sikerül neki kiszabadulnia, a hátralevő karanténból levonódik 1. A dobások után a dobókockák eltűnnek, ezzel utat engedve más interakcióknak. Amennyiben a dobásunknak a következménye, hogy a karakterünk elmozdul, így az addig lép ameddig az adott játékos mezőjének a kódjának a modulója 32-vel meg nem egyezik a dobás előtti mező kódja + a dobás összege moduló 32-vel. Például van egy játékos a 7-es mezőn, aki dob 4-et, akkor 7 MOD 32 = 7. Ez a 7 lesz a jelenlegi mező, amin a játékos áll, de a kívánt mezőt úgy számoljuk ki, hogy (7+4) MOD 32 = 11. Mivel a 7 nem egyenlő 11-gyel, így egyesével elkezdi a program növelni a játékos mezőjének értékét, egészen addig, ameddig a kívánt mezőre nem ér. Ezeknek a mezőknek a változását figyeli a program, és ennek hatására lép a karakter is. Ilyenkor ellenőrzésre kerül, hogy nem lépte-e át a játékos a kezdő mezőt, mert abban az esetben 250 000 játékpénzt kap.

6.1.2.4. Vásárlás és fejlesztés

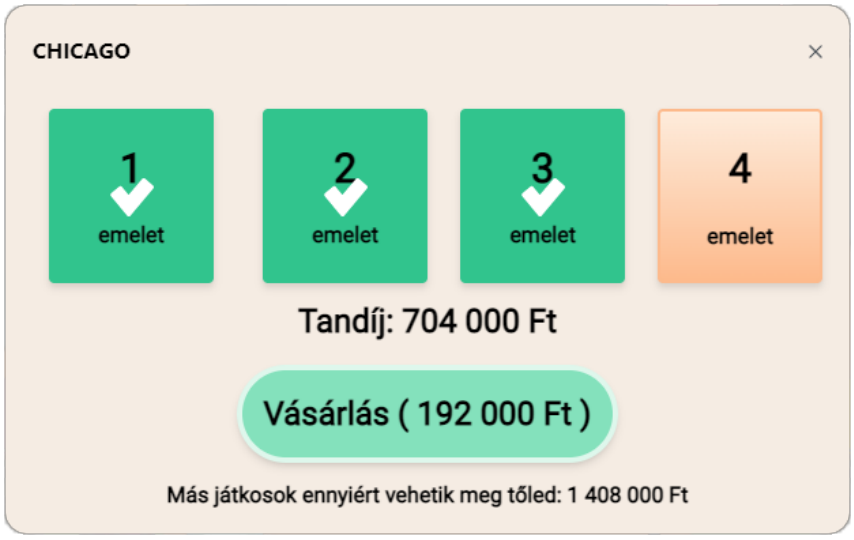
Amikor egy olyan mezőre lép egy játékos, amin valamilyen tulajdont lehet birtokolni, abban az esetben több feltétel is megvizsgálásra kerül. Először megnézi a program, hogy az adott területnek van-e már tulajdonosa és amennyiben nincs, akkor megvizsgálja, hogy lenne-e elegendő pénz a vásárláshoz. Amennyiben van,

felajánlja a lehetőséget a vásárlásra. Itt zölddel jelöli az elérhető szinteket, szürkével a nem elérhetőket és sárgával a kiválasztottakat.



6.8. ábra. Független mező vásárlása

Ha az adott területnek van már tulajdonosa, és az az adott játékos, akkor megvizsgálja a program a szintet, hogy van e továbbfejlesztési lehetőség. Ha van és van is rá pénz, akkor felajánlja a lehetőséget a fejlesztésre az alábbi felugró ablakkal.



6.9. ábra. Egyetem fejlesztése a maximum szintre

Az utóbbinál jól láthatóan a már birtokolt szinteknél sötétzöld színnel van jelölve a háttér és pipával van ellátva.

Viszont, ha nem az adott játékos birtokolja a mezőt, akkor kiszámításra és levonásra kerül a tandíj. Ha marad elég pénze a játékosnak arra, hogy megvegye ezek után az ingatlant, akkor a rendszer felajánlja a lehetőséget erre.



6.10. ábra. Más egyetemének megvétele

Ezeknél az interakcióknál jól látható a tandíj. Ez az érték az, amit más játékosok fognak fizetni, ha rálépnek az adott mezőre. Emellett kiemelve látszódik a vételi ár a vásárlási gombon zárójelek között, illetve a gomb alatt látható az is, hogy más játkosok mekkora összegért tudják megvenni az adott ingatlant. Az utóbbi mindig az egyetem értékének duplája.

Megvásárlást követően az ablak bezáródik, a pénz levonódik, majd a játék állapotát leíró objektumban, az adott mezőnek a szintjét beállítja a kiválasztottra, a tulajdonosát pedig arra, aki megvette. Ezeknek a paramétereknek a változásának hatására, megjelenik a megvásárolt egyetem vagy elszíneződik a múzeum.

6.1.2.5. Eladás

Az eladási procedúra akkor következik be, amikor egy játékosnak fizetési kötelezettsége van valaki felé, azonban azt nem tudja a meglévő pénzéből teljesíteni. Ekkor a játék paraméterei úgy változnak meg, hogy több saját mezőt is ki lehessen választani. Ez azért szükséges, mivel nem feltétlen tudja a játékos egy mező eladásával fedezni a kötelezettségét.

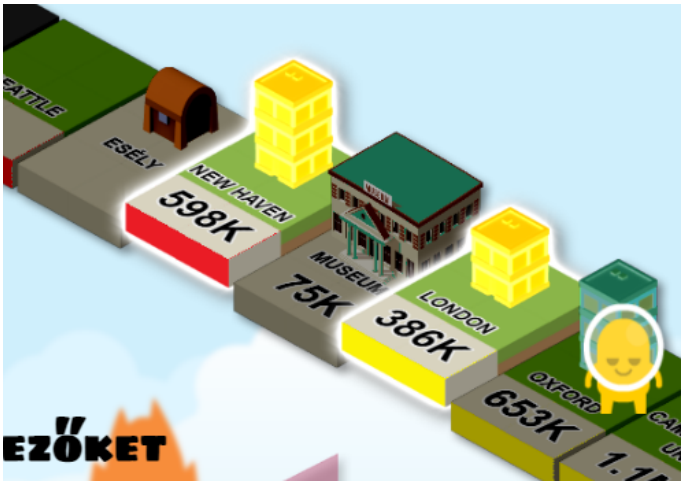


6.11. ábra. Eladási HUD megjelenik

Ekkor a felhasználónak 2 opciója van:

- Kiválasztja az eladni kívánt mezőket és rákattint az eladás gombra.
- Az automatikus eladást választja.

Az előbbinél láthatjuk, hogy megjelent középen az eladási rendszer. Ekkor valamely mezőinket ki kell választani, egészen addig, ameddig nem lesz elegendő a kiválasztott mezők eladási értéke.



6.12. ábra. Eladni kívánt egyetemek kiválasztva

A kiválasztott mezők elkezdenek ragyogni, így tudjuk megkülönböztetni a többitől. A kiválasztással egyidőben a korábbi képen látható HUD-on láthatjuk a

kiválasztott mezők értékét, illetve, hogy eladható-e, azaz eléri-e az eladási érték a kötelezettség értékét. Az utóbbi úgy számolódik, hogy a kiválasztott mezők kódjai egy tömbbe adódnak, a számoló algoritmus pedig figyeli ennek a változását, és ha változás történik, akkor automatikusan újraszámolja a tömb elemeire.



6.13. ábra. A mezők eladhatóvá váltak

Amennyiben az *ELADÁS* gomb zölddé válik, a játékos eladhatja a mezőket.

Ha nem a kiválasztós eladási módszert preferálja a felhasználó, akkor rá tud menni az *AUTOMATIKUS* gombra, ami alapján a mezők értékeik szerint növekvő sorrendbe rendeződnek és addig adódnak el azok, ameddig a játékos pénze eléri a kívánt mennyiséget.

6.1.2.6. Csőd

Amennyiben egy játékosnak fizetési kötelezettsége van egy másik játékos vagy a bank felé, azonban nincs rendelkezésre álló vagyon, akkor az a játékos csődbe kerül. Ez a következőképp jelenik meg játék közben:



6.14. ábra. A játékos csődbe kerülése

Ez logikailag úgy történik, hogy a rendszer érzékeli, hogy nem tudja teljesíteni a kötelezettséget, így összeszámolja a birtokolt területek eladási értékét, és ehhez hozzáadja a játékos pénzét. Ha ez az összeg kevesebb, mint ami szükséges, akkor a játékos csőd paraméterét beállítja igazra, majd a birtokolt területeket elveszi a játékostól és a kötelezettség értékét hozzáadja annak a pénzéhez, aki felé tartozik.

6.1.2.7. Köridő lejárása

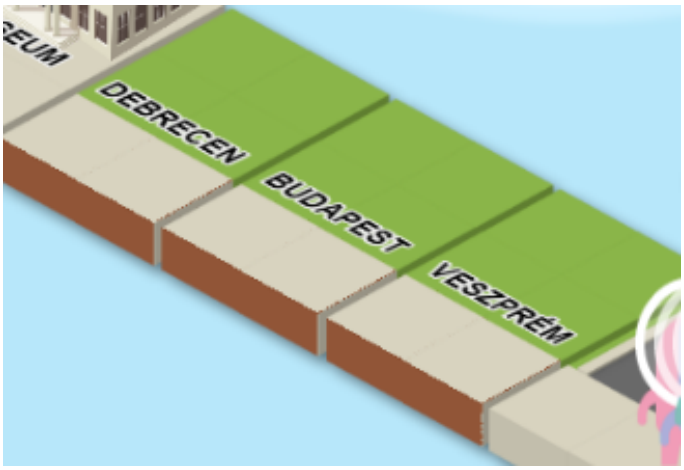
Amennyiben interakció közben lejár a rendelkezésre álló idő, az adott játékos büntetve lesz. Ez azt jelenti, hogy amennyiben nem tudja befejezni az interakciót, a pénzének 10%-a levonásra kerül. Azonban, ha az illető 100 000 játékpénz alá megy, akkor más történik. Itt megvizsgálja a rendszer, hogy van-e birtokolt tulajdon, ha van eladja és levon a pénzből 100 000-et. Azonban, ha nincs, akkor abban az esetben a játékos azonnal csődbe megy.

Ha a játékos valamilyen interakciót végzett, miközben lejárt az idő, akkor annak függvényében jár el a rendszer. Ha éppen eladás közben járt le az idő, akkor automatikusan eladja a szükséges mezőket, majd megbünteti a játékost. Amennyiben a játékos éppen adót köteles fizetni, de lejár az ideje, akkor levonja az adót, majd megbünteti a játékost. Ha a karanténban van az illető, akkor 1-gyel csökkenti a hátralevő karanténban töltendő körök számát.

6.1.2.8. Normál mező

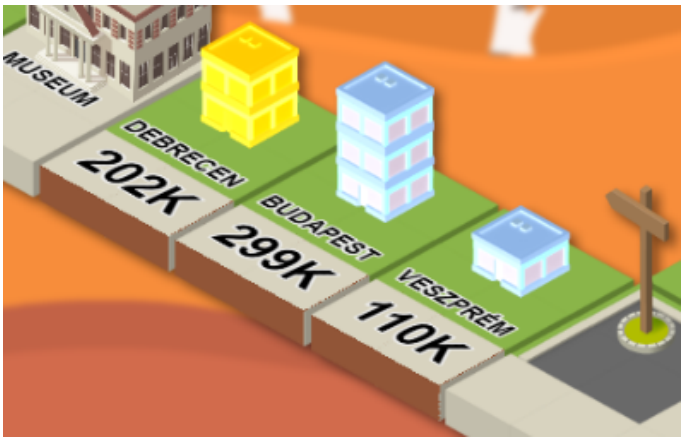
Normál mezőknek azok mondhatók, amiken egyetemeket lehet vásárolni. Ezek mind olyan városok neveivel vannak ellátva, ahol híresebb egyetemek találhatók. A mezők

mind hozzá vannak rendelve egy-egy csoporthoz. A mezők oldalán található szín jelzi az utóbbit. Ezeknek a rajta lévő egyetemek tandíjának kiszámítása, illetve a játék vége miatt van jelentősége, ugyanis, abban az esetben, ha egy játékos az adott csoport minden mezőjén birtokol egyetemet, akkor azokon a mezőkön a tandíj 40%-al megnő. Emellett, ha egy játékos 3 csoportnak minden területét birtokolja, akkor automatikusan megnyeri a játékot. Az utóbbi minden új kör előtt kerül ellenőrzésre.



6.15. ábra. Normál mezők alap kinézete

A tandíj a mezők nevei alatt találhatók rövid formában. Értelemszerűen a K az ezret jelenti, az M a milliót.



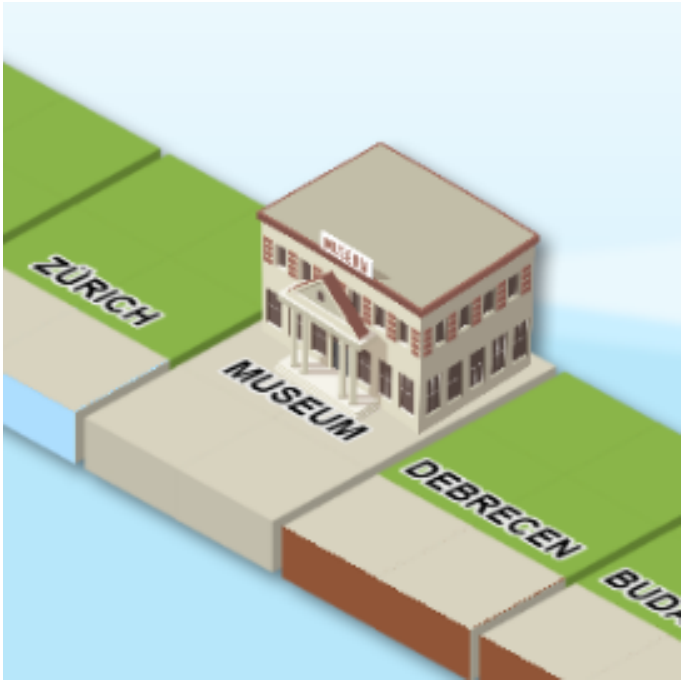
6.16. ábra. Normál mezők, rajta egyetemekkel

Ezen a képen jól láthatjuk, hogy ezeket a mezőket már birtokolják, mivel mind-

egyiken van egyetem. A színek alapján könnyen megállapítható, hogy ki a tulajdonos, illetve a magassága alapján az egyetem szintje is kikövetkeztethető.

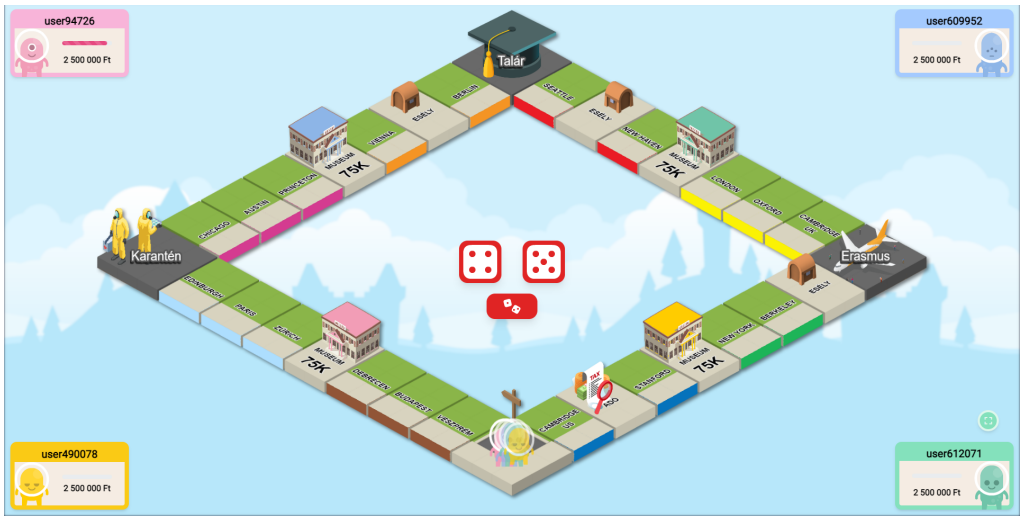
6.1.2.9. Múzeumok

Ezeken a mezőkön találhatók a múzeumok, amiket a játékosok meghatározott pénz ellenében megvásárolhatnak.



6.17. ábra. Múzeum alap kinézete

A múzeumok tetejei alapból megegyeznek a múzeumok más részeinek színével, azonban, ha valaki megvásárolja, akkor a teteje átváltozik arra a színre, ami a játékos színe, vizuálisan jelezve, hogy ki birtokolja az adott épületet.



6.18. ábra. Múzeumok, amiket birtokolnak

Az előbbi képen láthatjuk, hogy a múzeumokat mind más és más birtokolja, így a színük is különböző.

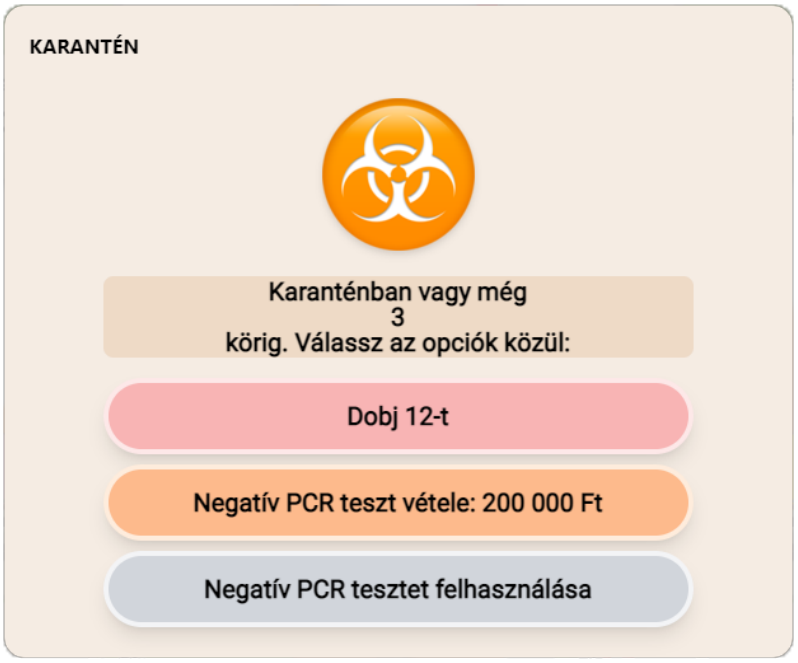
6.1.2.10. Karantén

A karantén mező a bal sarokban helyezkedik el. Ennek a lényege a játékosok akadályozása. Amennyiben valaki rálép, az alábbi ablak jelenik meg számára:



6.19. ábra. Karanténba kerülés

Ekkor beállítódik az adott játékosra, hogy karanténban van, illetve, hogy 3 hátralevő körből kimarad. Ezek után, ha újra rákerül a sor, megpróbálkozhat a kijutással.



6.20. ábra. Karanténban látható felület

A kijutásra 3 lehetőség van:

- Az első, hogy a játékos dobhat, ekkor beállítódik, hogy a dobás karanténból történik, a panel eltűnik és megjelenik a dobókocka. Ha 12-t dob az illető, akkor kiszabadul, viszont, ha ez nem sikerül, akkor a hátralevő karanténban töltött körök száma csökken egyet és a következő játékos következik
- A második lehetőség, hogy 200 000 játékbeli pénzért tudunk venni negatív PCR tesztet, akkor az összeg levonódik a játékos pénzéből és kiszabadul a karanténból, majd egyszerre dobhat is.
- Az utolsó lehetőség nem más, mint az, ha már rendelkezünk negatív PCR teszttel, akkor felhasználhatjuk azt. Ekkor a játékostól elvételre kerül a teszt és már léphet is. Negatív PCR tesztet az esély mezőre lépést követően lehet kis szerencsével szerezni.

Ha valamely opció nem érhető el, akkor annak a háttere beszürkül, és nem lehet rákattintani. Emellett minden egyes körben látni, hogy hány körig van a játékos karanténban.

6.1.2.11. Talár

A talár mező a tetején található a játéknak. Ez a játékosok segítéséül szolgál, mert aki rálép az választhat a tulajdonai közül, hogy melyiken szeretné megemelni a tandíjat.

Miután a mezőre rálép valaki, a program ellenőrzi, hogy van e birtokában egyetem vagy múzeum. Amennyiben nincs, semmi nem történik, a következő játékos dob. Viszont, ha a jelenlegi játékosnak van, abban az esetben felhozza az alábbi ablakot:



6.21. ábra. Talár

Itt a gombra kattintva, megkezdődik a kiválasztás. Ekkor a játék állapotát leíró objektumban átváltoznak azok a logikai paraméterek, amik hatására megkezdődik a saját tulajdon kiválasztása. Ez a következőképp néz ki:



6.22. ábra. Talár hatására mező kiválasztása

Ekkor az adott mező paraméterei átíródnak, beállítódik, hogy van rajta talár, illetve, hogy hány talár került már rá eddig. Ennek következtében a tandíj az adott mezőn megduplázódik. A talár többször is rárakható egy mezőre, ekkor a duplázó értéke nélküli árral nő a tandíj.



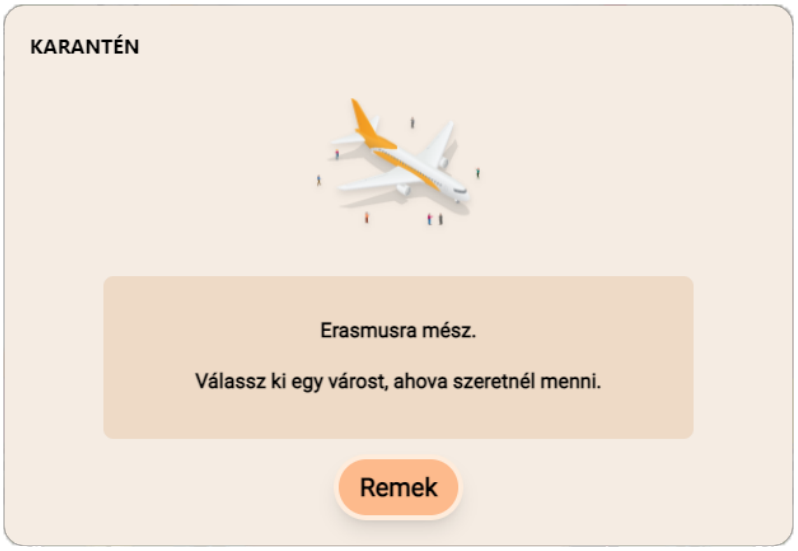
6.23. ábra. Talár a múzeumon

Jól láthatóan egy platform került a múzeum mögé egy talárral, így jelezvén, hogy

melyik mező van a duplázó hatása alatt.

6.1.2.12. Erasmus

Az Erasmus mező a játék jobb sarkában található. Célja, hogy ha a játékos rálép, akkor ki tudja választani, hogy melyik mezőre szeretne lépni. Rálépést követően történik egy logikai vizsgálat, ami azt elemzi, hogy létezik-e olyan mező amire rá tud lépni a játékos. Ugyanis csak saját tulajdonú vagy független normál, illetve múzeum mezőre lehet lépni. Ezt követően ugrik fel az alábbi tájékoztató jellegű ablak:



6.24. ábra. Erasmus mezőre lépés után felugró ablak

Ezután a játék paraméterei úgy állítódnak, hogy egy független vagy saját mezőt ki lehessen választani.



6.25. ábra. Erasmus - mező kiválasztása

Ekkor a felhasználó kiválasztja, hova szeretne lépni, majd a karakter odalépked az alapján, hogy a program kiszámolja a két mező közti különbséget, majd átírja a játékos mezőjét a kiválasztott mezőre. A kiválasztás ezzel párhuzamban eltűnik, és azután lehet más interakciókat végezni, mint például vásárlás vagy fejlesztés.

6.1.2.13. Adó

Az adó mező az utolsó előtti mező. Rálépést követően a birtokolt épületek értékeinek 10%-át adóként be kell fizetni. Itt is először egy logika vizsgálat indul, ami megvizsgálja, hogy a játékosnak van-e elegendő pénze. Amennyiben van, az összeg levonódik, de ha nincs, akkor az eladási procedúra veszi kezdetét. Mindezekről egy információs ablak tájékoztatja a játékost.



6.26. ábra. Adózás

Itt látható, hogy pontosan mennyi összeget kell befizetni.

6.1.2.14. Esély

Az esély mezőből 3 darab található a játékban. Ezek véletlenszerű kártyákat adnak a játékosnak, amik lehetnek jók, illetve rosszak. Mezőre lépés után a rendszer véletlenszerűen választja ki a kártyát és annak megfelelően mutatja az alábbi ablakot:



6.27. ábra. Esély kártya

Jelenleg a szakdolgozat keretein belül 3 darab esély kártya került implementálásra,

amik az alábbiak:

- *"Nyertél egy negatív PCR tesztet, amit legközelebb felhasználhatsz, ha karanténba jutsz."*
- *"Olyan jó átlagod lett, hogy 200 000 ft ösztöndíjban részesülsz."*
- *"Sajnos minden tárgyadból megbuktál, így 200 000 ft-ot kell fizetned, hogy újra felvehesd őket."*

Az esélykártyáknak is megvannak a saját paramétereik, amik a következők:

- id: kódja a kártyának
- PCR: negatív PCR-t ad-e
- money: pénzt ad-e vagy von le
- moneyValue: mennyi pénzt ad vagy von le
- title: felugró ablak címe
- desc: kártya leírása

Ezen tulajdonságok függvényében változnak az adott játékos paraméterei.

6.1.2.15. Websocket események

Az alábbi táblázat a szobákhoz tartozó websocket eseményeket tartalmazza, illetve azok rövid leírását. Ezek mellett megtalálható benne, hogy kliens vagy szerver oldalról történik-e az esemény küldése.

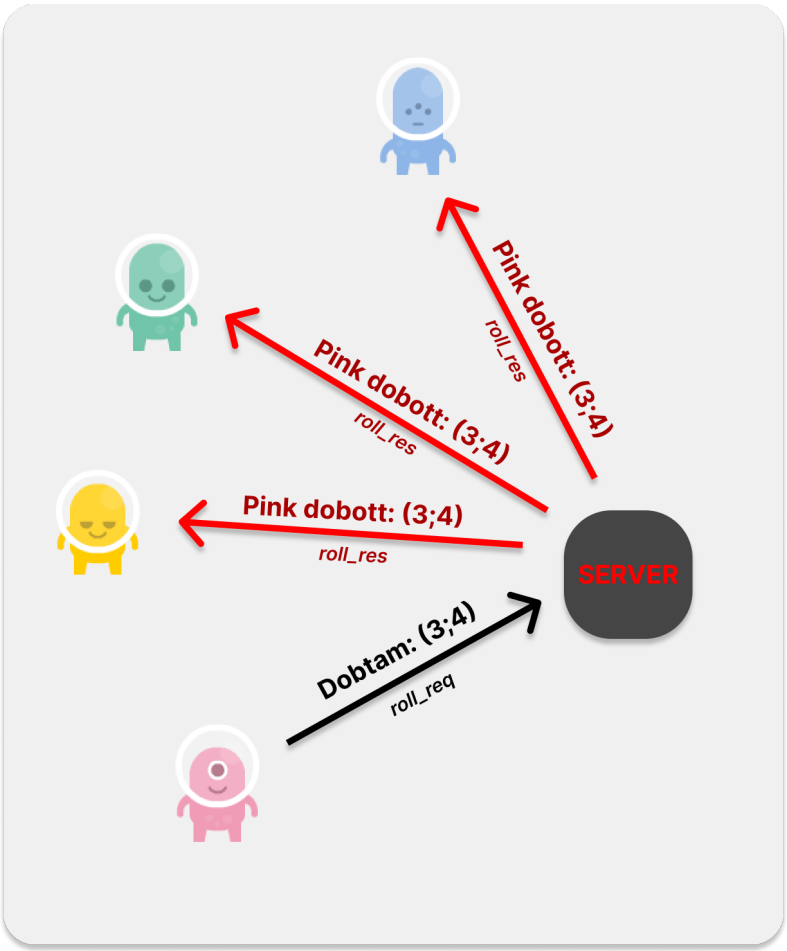
Event név	Leírás	Küldő
get_game_data_by_code_req	lekérdezi a játék kezdő adatait	kliens
get_game_data_by_code_res	megkapja a játék kezdő adatait	szerver
roll_req	elküldi a dobást a játékosoknak	kliens
roll_res	megkapják a dobást	szerver
step_on_field_controller_req	elküldi melyik mezőre lépett	kliens
step_on_field_controller_res	megkapják melyik mezőre lépett	szerver
buy_sajat_req	a fejlesztést elküldi	kliens
buy_sajat_res	a fejlesztést megkapják	szerver

Event név	Leírás	Küldő
buy_nem_sajat_req	a vásárlást elküldi	kliens
buy_nem_sajat_res	a vásárlást megkapják	szerver
tax_req	elküldi az adózás végrehajtását	kliens
tax_res	megkapják, hogy adózik	szerver
museum_buy_req	múzeum vételét elküldi	kliens
museum_buy_res	múzeum vételét megkapják	szerver
close_chance_req	esély bezárását elküldi	kliens
close_chance_res	esély bezárását megkapják	szerver
set_selected_card_req	elküldi melyik kártya sorsolódott ki	kliens
set_selected_card_res	megkapják a kisorsolódott kártyát	szerver
quarantine_roll_handler_req	elküldi, hogy megpróbál 12-t dobni	kliens
quarantine_roll_handler_res	megkapják, hogy megpróbál 12-t dobni	szerver
quarantine_buy_handler_req	elküldi, hogy vesz negatív tesztet	kliens
quarantine_buy_handler_res	megkapják, hogy vesz negatív tesztet	szerver
use_pcr_req	elküldi, hogy felhasznál egy tesztet	kliens
use_pcr_res	megkapják, hogy felhasznál egy tesztet	szerver
quarantine_close_req	elküldi a karantén ablak bezárását	kliens
quarantine_close_res	megkapják a karantén ablak bezárását	szerver
on_autoSell_handler_req	elküldi, hogy automatikus eladja	kliens
on_autoSell_handler_res	megkapják, hogy automatikusan eladja	szerver
on_sell_handler_req	elküldi, hogy eladás történt	kliens
on_sell_handler_res	megkapják, hogy eladás történt	szerver
select_field_req	elküldi a kiválasztott mezőt	kliens
select_field_res	megkapják a kiválasztott mezőt	szerver
doubler_close_req	talár bezárását elküldi	kliens
doubler_close_res	talár bezárását megkapják	szerver
erasmus_close_req	erasmus bezárását elküldi	kliens
erasmus_close_res	erasmus bezárását megkapják	szerver
time_over_req	idő lejárásának elküldése	kliens
time_over_res	idő lejárásának megkapása	szerver

6.2. táblázat. Játékhoz tartozó eventek

A játék úgy van implementálva, hogy amennyiben valaki valamilyen interakciót

végez, akkor azt végrehajtja és elküldi az interakció eventjét websocketen és ezt az elküldött eventet a szerver tovább küldi a többi játékosnak, akik megkapva végrehajtják ugyanazt az interakciót a saját játékukban. Például a pink játékos dob az egyik kockával 3-at, a másikkal 4-et. Ekkor a pink játékosnak az interakció végrehajtódik, majd ezt elküldi a szervernek, ahol a szerver a többi játkosnak is elküldi. A többi játékosok azt kapják meg, hogy a pink játékos dobott az egyik kockával 3-at, a másikkal 4-et, majd ennek megfelelően minden játékos a saját játékának állapotát eszerint állítják be. Az utóbbit szemléltetve:



6.28. ábra. Dobás küldése websocketen

Így, hogy minden változás kliens oldalon történik, nagyon nagy pontosságot igényel, ugyanis, ha valamelyik játékosnál máshogy, vagy más sorrendben történik meg a játék állapotának a beállítása az interakciónak megfelelően, akkor sajnos a játék hamar kiesik a szinkronból. Ez úgy küszöbölhető ki, hogy az állapotok módosítása

nem annyi helyen történik, ahány játékos játszik, hanem egy helyen. Ez azt jelenti, hogy az állapot változást szerver oldalon kell elvégezni, majd azt kiküldeni a játékosoknak. Ennek a javítása már csak szakdolgozat keretein kívül valósul meg.

6.1.2.16. Grafikai elemek

A játék során számos grafikai elem került felhasználásra. Ezek viszont külső forrásból származó tartalmak. A grafikai elemek többsége CC0 (Creative Commons Zero) [20] lincenszel rendelkezik. A játékban az alábbi grafikai elemeket KenneY [21] készítette:

- mezők
- karakterek
- hátterek
- épületek
- bizonyos ikonok
- dobókockák
- esély ládák

A további grafikai elemek a freepik.com [22] oldaláról származnak:

- fatábla, visszaszámoló (ismeretlen)
- start mezőn lévő nyíl (ismeretlen)
- múzeumok (macrovector)
- karantén mezőn a tesztelők (macrovector)
- talár (macrovector)
- Erasmus mezőn a repülő (macrovector)
- adó ikon (macrovector)

6.2. Tesztek

A játék természetesen többszörösen le lett tesztelve. A tesztekben 9 személy vett részt magamat is beleszámolva, akikkel a játékok után egy űrlapot töltöttem ki, hogy a visszajelzésüket és véleményüket rögzíteni tudják. A teszt 3 tesztelési fázisból állt.

6.2.1. Első tesztelés

Az első teszt során fény derült olyan hibákra, mint például manuális eladási lehetőség nem volt elérhető és csak automatikusan lehetett eladni a tulajdonokat. Ez a tesztelés 2 játékból állt. Ezek után a talált hiba javításra került és kezdődött a második tesztelés.

6.2.2. Második tesztelés

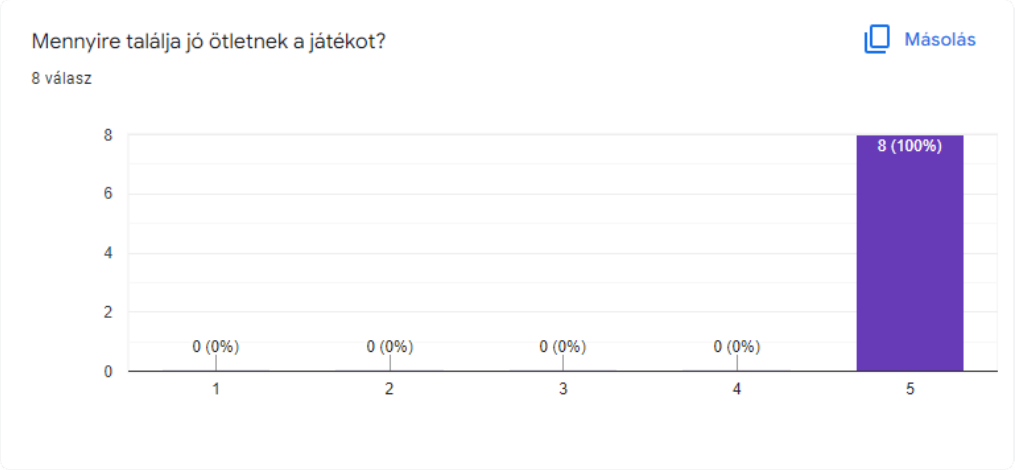
Ez a tesztelés 5 játékot tartalmazott, ami azért volt jó, mert rengeteg szituációt lehetett tesztelni. Ennyi játék mellett csak egyetlen egy hibát találtak a játékosok, ami pedig az volt, hogy 3 csoport birtoklása után nem ért véget a játék. Ezt követően ez a hiba is javításra került, és kezdődött a harmadik tesztelés.

6.2.3. Harmadik tesztelés

Ekkor 7 játék került végigjátszásra, hiba nélkül. Igaz a játék még így is csak egy alpha verziónak felel meg, viszont ismert hiba nincs a szoftverben.

6.2.4. Visszajelzések

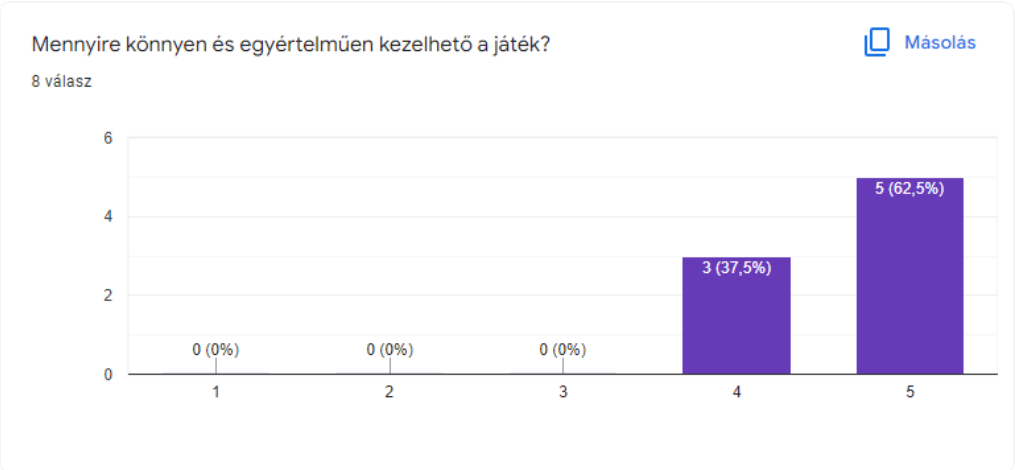
6.2.4.1. Mennyire találja jó ötletnek a játékot?



6.29. ábra. 1. visszajelzés

Érdekelt, hogy a játékosok mit válaszolnak erre a kérdésre, de egyhangúan mindenki nagyon jó ötletnek találja ezt a játékot.

6.2.4.2. Mennyire könnyen és egyértelműen kezelhető a játék?

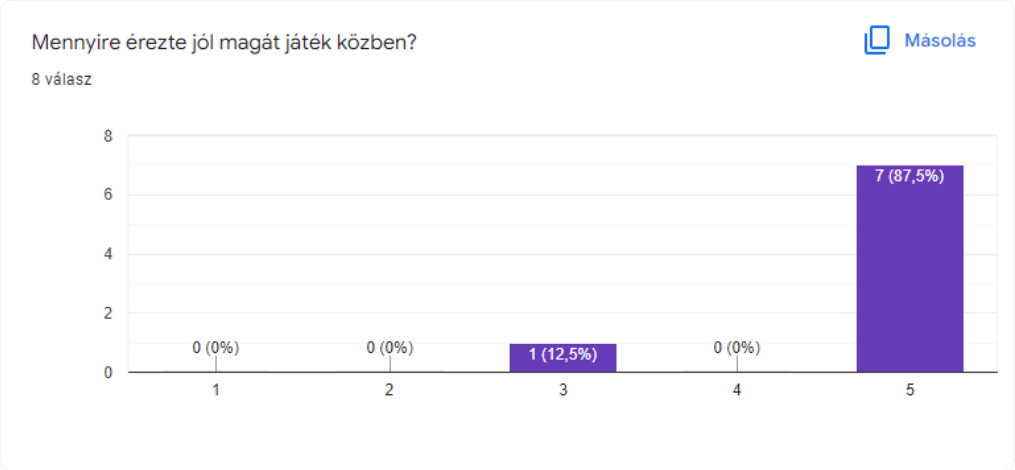


6.30. ábra. 2. visszajelzés

A tesztben részvevő játékosok egyáltalán nem lettek tájékoztatva arról, hogy a játékot hogyan kell kezelni, pont annak érdekében, hogy itt releváns válaszok szü-

lessenek. Szerencsére mindenki könnyedén rájött, hogyan kell használni.

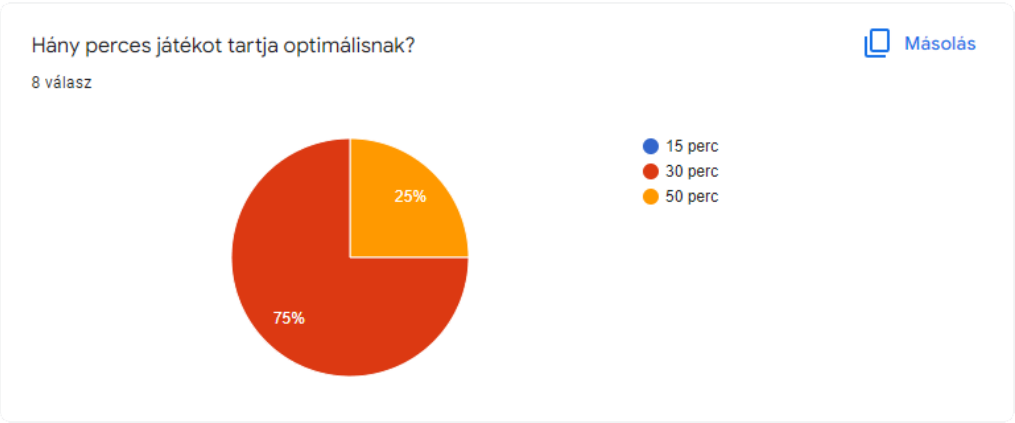
6.2.4.3. Mennyire érezte jól magát játék közben?



6.31. ábra. 3. visszajelzés

Itt is elég egyhangú a válasz egyet leszámítva. A játékos előre közölte velem, hogy nem szereti a társasjátékokat, de szívesen segít leteszteltni. Ezek mellett utolsó lett a játék folyamán, így a közepes válasz ennek tudható be.

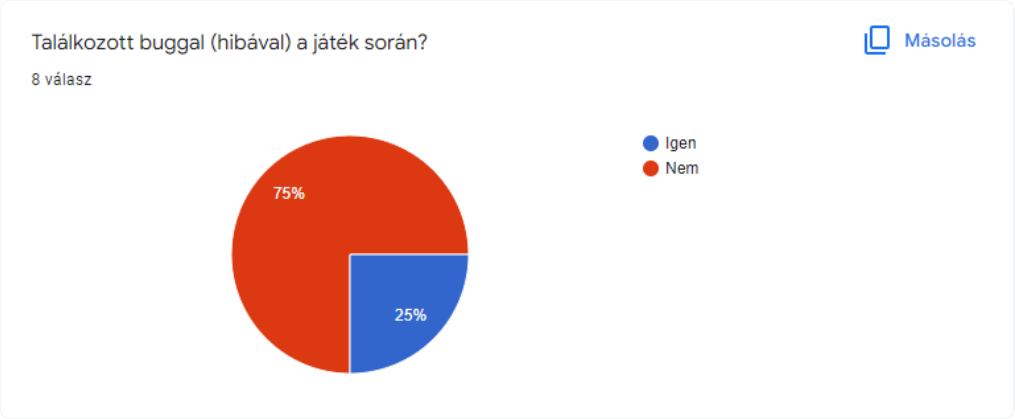
6.2.4.4. Hány perces játékot tartja optimálisnak?



6.32. ábra. 4. visszajelzés

A tesztelők 75%-a szerint a 30 perc pont ideális, 25% pedig inkább a hosszabb játékmeneteket élvezik.

6.2.4.5. Találkozott buggal (hibával) a játék során?



6.33. ábra. 5. visszajelzés

A 8 tesztelőből 2 jelentette, hogy talált hibát. Ezeket korábban fejtettem ki és ezek már ki lettek javítva.

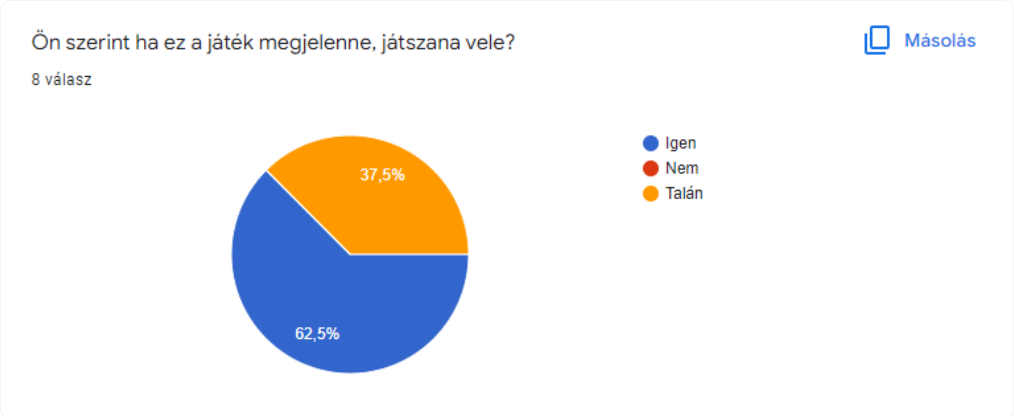
6.2.4.6. Hiányolja a hangokat, illetve háttérzenét a játék során?



6.34. ábra. 6. visszajelzés

Számomra ez egy meglepő arány, mivel szinte biztosra vettem, hogy hiányozni fognak a hangeffektek, illetve a háttérzene, viszont ezeket csak a tesztelők 25%-a hiányolta.

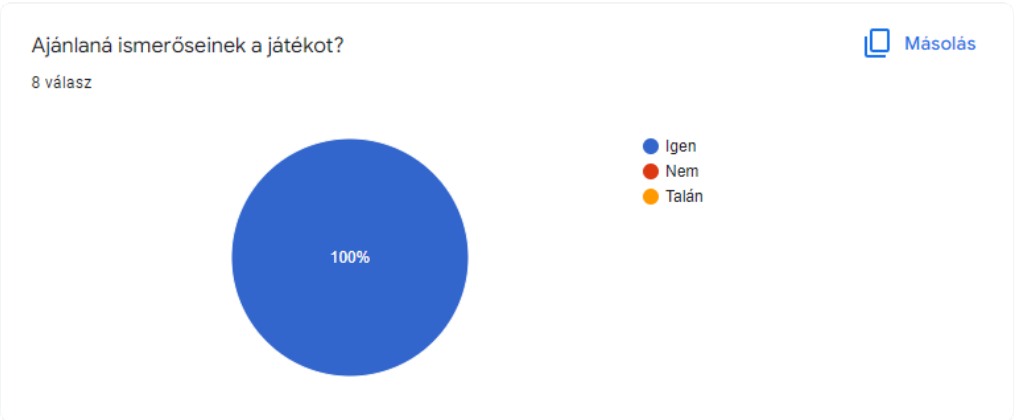
6.2.4.7. Ön szerint, ha ez a játék megjelenne, játszana vele?



6.35. ábra. 7. visszajelzés

Itt a többség (62%) egyértelműen játszana vele, ha ez a játék megjelenne és a maradék sem vetette el ennek az esélyét. Szerencsére senki nem érezte magát annyira rosszul játék közben, hogy ne szeretne vele többet játszani.

6.2.4.8. Ajánlaná ismerőseinek a játékot?



6.36. ábra. 8. visszajelzés

6.2.4.9. Ha bármilyen észrevétele vagy ötlete van a játékkal kapcsolatban kérem írja le.

Ezt a kérdést a későbbi felhasználói élmények javítása miatt tettem fel, mivel fontos számomra, hogy a felhasználók minél jobban érezzék magukat, mivel a játék pont ezért jött létre. Itt az alábbi ötleteket írták a tesztelők:

"Körülbelül 10 perc után, akinek a legtöbb tulajdona van, meg lehetne fosztva a körönkénti 250ezer bónusztól" - Ezt nem tartom rossz ötletnek, mivel ezáltal egy kiegyensúlyozottabb játékmenet jöhetne létre, ezt a későbbiekben mindenképpen meg kell vizsgálni.

"Feladás gomb egy jó lehetőség lenne, abban a helyzetben, ha már csak 2 játékos van és előre eldönthető a játék kimenetele." - Ez is egy jó ötlet, mivel ezáltal nem kellene feleslegesen még percekig lépkedni, ha már eldönthető a játék végkimenetele.

"Lehetnének a figurák különböző diák típusokról mintázva (pl.: a bulizós, a stréber...)" - Jelen fázisában a játék ingyenes grafikai elemekből épül fel, és ennyire specifikus karaktereket nem találni. Ezek a későbbiekben egyéni grafika készítésével lehetnének megoldva.

7. fejezet

Szoftver

7.1. A szoftver telepítése és futtatása

7.1.1. Szükséges programok

A szükséges programok a szoftver telepítésére és futtatására:

- node.js
- npm
- yarn

7.1.2. Telepítés és futtatás

A mellékletben található software mappa tartalmát először ki kell csomagolni. Ezt követően a client mappában telepíteni kell a modulokat. Ehhez a client mappában ki kell adni a 'yarn' parancsot. Amint ez kész, ugyanezt elvégezzük a server mappában.

Ezt követően futtatni kell a szervert és a klienset is. Ehhez a server mappában ki kell adni a 'yarn start' parancsot. Az előbbit nem szabad bezárni. Ezt követően a client mappában ki kell adni a 'yarn game' parancsot. Amennyiben elindítottuk a szervert, az a localhost 3001-es portján fog futni, a kliens pedig a localhost 3000-es portján. A kliens automatikusan a localhost 3001-es portján keresi a szervert. Amennyiben ez máshol található, a client/src/config.js fájl első sorában a 'localhost:3001'-t át kell írni '{szerver elérési útja}'-ra. Viszont amennyiben ezt átírtuk, a client mappában ki kell adni a 'yarn build' parancsot és csak ezután elindítani a klienst a 'yarn game' paranccsal. A 'yarn build' a játék fájljainak buildelését végzi el.

8. fejezet

További lehetőségek/továbbfejlesztési tervek

A későbbiekben szeretném továbbfejleszteni a játékot, mivel számomra ennek a fejlesztése egy hosszabb távú projekt. A játék jelen formájában úgy van megírva, hogy könnyen skálázható legyen, erre külön figyeltem, hogy a későbbi fejlesztések során könnyen, gyorsan és produktívan lehessen haladni úgy, hogy szükség esetén az eddig elkészült kódot csak minimálisan kelljen módosítani.

8.1. Authentikáció

A későbbiekben szeretnék egy authentikációt beleépíteni, ahol lehet regisztrálni, illetve belépni email címmel. Emellett bizonyos social platformokon (Facebook, Google, Twitter) keresztül történő authentikáció támogatása is bekerül az alkalmazásba és ezáltal hozzáférhetünk az addigi elért eredményeinkhez. Az alapvető authentikációs módszert, a JWT-t (JSON Web Token) használva lesz megvalósítva.

8.2. Lokális multiplayer

A belépési rendszer mellett fontosnak tartom azt, hogy támogatva legyen a lokális multiplayer. Ez azt jelenti, hogy amennyiben a felhasználók ugyanazon a helyen tartózkodnak, akkor elég egy eszköz, hogy játszhassanak és nem kell mindenkinek saját eszközzel rendelkeznie. Ilyenkor a játékmenet az adott eszközön történik és a játékosok felváltva kezelik azt, annak megfelelően, hogy éppen kire kerül a sor.

8.3. Hangok és zene

Manapság már szinte nem is találni olyan játékot, ami nem tartalmaz hangeffekteket és/vagy zenét, ami szól a háttérben. Úgy gondolom, hogy a játékélményt jelentős mértékben növelné, ha minden interakciót követően a felhasználó kapna auditív visszajelzést annak megfelelően, hogy milyen típusú cselekvés történt.

Belátásom szerint a jól kiválasztott vidám háttérzene megalapozza a játék alaphangulatát és esetenként a játék okozta csalódottságot, illetve negatív hatásokat is enyhíteni tudja.

8.4. Esély kártyák bővítése

A még jobb játékélmény elérésének érdekében tervben van az esély kártyák paklijának bővítése. Ezáltal sokszínűbbé téve a játékot, komolyabb taktikai elemeket létrehozva, mint például egyetem elpusztítása, vagy Erasmus ösztöndíj elnyerése. Így sokkal kevesebb lesz a kártyák ismétlődésének az esélye is.

8.5. Fizetős tartalom

Hosszútávú terveim között szerepel bizonyos fizetős tartalom bevezetése. A játék azt követően is ingyenes és reklámmentes marad, de kerülnek bele olyan elemek, amiket majd csak igazi pénzért megvásárolt tokenekért tudunk megszerezni. Ilyenek például a karakter öltözékek, vagy dobókocka kinézetek. Ezen felül játék közben használható segítség is elérhető lesz ezekért a tokenekért, de annak érdekében, hogy ne befolyásolja olyan mértékben a játékot, hogy jóformán pénzzel meglehet venni a győzelmet, így minden nap le lesz korlátozva, hogy maximum ezekből a segítségekből mennyit lehet felhasználni.

8.6. Matchmaking

A matchmakingnek az a lényege, hogy a felhasználó úgy szeretne játszani, hogy nem a szobákban szeretne belépegetni ahhoz, hogy játszon, hanem csak szeretne egy

gombra rákattintani és onnantól maga a rendszer összepárosítja másik játkosokkal és létrehozza a játékot. Ez egy sokkal könnyebb és egyszerűbb módja azoknak a személyeknek, akik ismeretlenekkel szeretnének játszani.

8.7. Telefon, tablet támogatás

Jelen fázisában a programnak, ha telefonról vagy tabletről próbálunk játszani, akkor azt tapasztalhatjuk, hogy a pálya nem tartja meg az arányait és eltolódik vagy elnyúlik valamely irányba. Ennek a kijavítása lenne a cél, hogy mindenki tudjon játszani bármilyen platformról.

8.8. Alkalmazás és applikáció

Távoli cél még többek között, hogy a játékból készüljön Androidra és iOS-re is egy alkalmazás, illetve számítógépre is lehessen telepíteni natív alkalmazásként.

8.9. Más nyelvek támogatása

A szoftverben jelenleg is implementálva és támogatva van a többnyelvűség, de még a szövegek nincsenek teljes mértékben lefordítva, így ez jelenleg nem elérhető. A későbbiekben le lesz fordítva az oldal angol, illetve német nyelvre. Ez azért fontos cél, mivel nemzetközi szinten játszható játékká szeretném tenni. Majd a távoli jövőben a remélhetőleg a lelkes játékosbázis segít az oldal további lefordításában, így valóban azokon a nyelveken lesz elérhető a játék, amilyen nyelveket beszélő játékosok játszanak vele.

9. fejezet

Összefoglalás

Szakdolgozatom elkészítése egy érdekes kaland volt számomra, ami alatt sokat tanulhattam egy folyamatosan fejlődő területről. Kezdetben sok utánajárást igényelt a téma. Fantasztikus érzés volt olyan technológiákkal dolgozni, amik a legpopulárisabbak és legkeresettebbek manapság a webfejlesztés világában. Örülök, hogy ezeket megtanultam a dolgozat keretein belül, mivel ezt a tudást a későbbiekben is felhasználhatom. Emellett sokat javult a problémamegoldó képességem. A dolgozat során megtapasztalhattam milyen egy nagyobb feladaton huzamosabb ideig dolgozni. Játék fejlesztése egy számomra eddig soha nem próbált, illetve nem ismert terület volt. Izgalmas volt a játék minden elemét kódsoronként megírni mindenféle modern játékmotor használata nélkül. Majdnem, hogy a játékmenet kitalálása okozta a legnagyobb fejtörést, mivel egy olyan témát szerettem volna neki adni, amilyet még sehol nem láttam. Így jött az ötlet az egyetemes témához és kerültek bele olyan elemek is, amik a mindennapi problémáinká nőtték ki magukat az elmúlt években. A játék viszont abból a szempontból könnyen továbbfejleszthető, hogy egyszerűen lehet minimális módosításokkal más témát adni neki. A szobarendszere letisztult és felhasználóbarát, ezért könnyű kezelni. Ezt a későbbiekben a játéktól függetlenül is lehet használni más-más programok keretein belül. Úgy érzem sikerült egy olyan játékot létrehozni, ami már most sok embernek örömet és izgalmas perceket vagy órákat okozott. A szakdolgozatom elkészítésével belekezdtem egy számomra nagyobb projekt megvalósításába, amit ezek után már biztosan szeretnék folytatni, mivel így a nagy része már implementálásra került. Remélem, hogy ezt a játékot játszva a jövőben nagyon sokan legalább olyan jól fogják magukat érezni, mint én miközben készítettem és láttam, ahogy az apró elemek elkezdenek egy nagyobb és komplex rendszert alkotni és végül pedig játékká válni.

Irodalomjegyzék

[1] „Node.js, url: https://nodejs.org/." (Utoljára látogatva 2021. 09. 23.).

[2] „Express, url: https://expressjs.com/." (Utoljára látogatva 2021. 09. 23.).

[3] „Socket.io, url: https://socket.io/." (Utoljára látogatva 2021. 09. 23.).

[4] „React js, url: https://reactjs.org/." (Utoljára látogatva 2022. 03. 05.).

[5] „Pixijs, url: https://pixijs.com/." (Utoljára látogatva 2022. 04. 28.).

[6] „Google play - monopoly - classic board game, url: https://play.google.com/store/apps/details?id=com.marmalade.monopoly&hl=en_US." (Utoljára látogatva 2021. 10. 12.).

[7] „App store - monopoly - classic board game, url: https://apps.apple.com/us/app/monopoly/id1477966166." (Utoljára látogatva 2021. 10. 12.).

[8] „Business tour - board game with online multiplayer, url: https://store.steampowered.com/app/3
97900/Business_Tour__Board_Game_with_Online_Multiplayer/." (Utoljára látogatva 2021. 10. 12.).

[9] „Y8 - monopoly, url: https://www.y8.com/games/monopoly." (Utoljára látogatva 2021. 10. 12.).

[10] Ubisoft, „Monopoly, url: https://www.ubisoft.com/en-us/game/monopoly/monopoly." (Utoljára látogatva 2021. 10. 12.).

[11] Stadia, „Monopoly, url: https://stadiasource.com/game/59/monopoly." (Utoljára látogatva 2021. 10. 12.).

[12] „skribbl.io, url: https://skribbl.io/." (Utoljára látogatva 2021. 10. 12.).

[13] „Balsamiq wireframes, url: https://balsamiq.com/wireframes/." (Utoljára látogatva 2021. 11. 16.).

[14] „Figma, url: https://www.figma.com/." (Utoljára látogatva 2022. 05. 07.).

[15] „diagrams.net, url: https://www.diagrams.net/." (Utoljára látogatva 2022. 05. 06.).

[16] „Material icons , url: https://mui.com/material-ui/material-icons/." (Utoljára látogatva 2022. 04. 20.).

[17] „Mantine , url: https://mantine.dev/." (Utoljára látogatva 2022. 04. 20.).

[18] „Tailwind css , url: https://tailwindcss.com/." (Utoljára látogatva 2022. 04. 25.).

[19] „i18next, url: https://www.i18next.com/." (Utoljára látogatva 2022. 01. 18.).

[20] „Creative commons zero, url: https://creativecommons.org/publicdomain/zero/1.0/." (Utoljára látogatva 2022. 03. 23.).

[21] „Kenney, url: https://www.kenney.nl/." (Utoljára látogatva 2021. 03. 23.).

[22] „freepik oldala, url: https://www.freepik.com/." (Utoljára látogatva 2021. 04. 02.).

[23] „Kenney, url: https://www.kenney.nl/." (Utoljára látogatva 2021. 03. 23.).

10. fejezet

Mellékletek

10.0.0.1. Beadott fájlok

2122_2_SzenteDániel_A1A9CA_Szakdolgozat_VP/software mappa (a szoftver fájljai):

client.zip	A kliens fájljai tömörítve
server.zip	A szerver fájljai tömörítve

2122_2_SzenteDániel_A1A9CA_Szakdolgozat_VP/tex mappa (tex fájlok és képek tömörítve):

a1a9ca_tex.zip	A szakdolgozat tex fájljai tömörítve

2122_2_SzenteDániel_A1A9CA_Szakdolgozat_VP/forrasok mappa (internetes források, a fájlnevek megfelelnek az irodalomjegyzékbeli sorszámaiknak):

1.mht	13.mht
2.mht	14.mht
3.mht	15.mht
4.mht	16.mht
5.mht	17.mht
6.mht	18.mht
7.mht	19.mht
8.mht	20.mht
9.mht	21.mht
10.mht	22.mht
11.mht	23.mht
12.mht	

Ábrák jegyzéke

3.1. React, Angular és Vue letöltései az elmúlt évben az npmtrends [23] oldalán 14
3.2. Express és hasonló keretrendszerek letöltései az elmúlt évben az npmtrends [23] oldalán 15

5.1. Egyedileg összeállított színpaletta 21
5.2. Oldaltérkép 22
5.3. Architektúra 24
5.4. A főoldal látványterve 25
5.5. A nyelvválasztó menü látványterve 26
5.6. A szobák látványterve 26
5.7. Szoba létrehozásának látványterve 27
5.8. Szoba látványterve 28
5.9. Játék látványterve 29
5.10. A ranglista látványterve 30

6.1. A főoldal 31
6.2. Szobák listázása 33
6.3. Új szoba létrehozása 34
6.4. Szoba 35
6.5. Nyelvválasztó menü 36
6.6. Kezdő állapot a mezők generálása után 40
6.7. Dobókocka dobás után 41
6.8. Független mező vásárlása 42
6.9. Egyetem fejlesztése a maximum szintre 42
6.10. Más egyetemének megvétele 43
6.11. Eladási HUD megjelenik 44
6.12. Eladni kívánt egyetemek kiválasztva 44

6.13. A mezők eladhatóvá váltak 45
6.14. A játékos csődbe kerülése 46
6.15. Normál mezők alap kinézete 47
6.16. Normál mezők, rajta egyetemekkel 47
6.17. Múzeum alap kinézete 48
6.18. Múzeumok, amiket birtokolnak 49
6.19. Karanténba kerülés 49
6.20. Karanténban látható felület 50
6.21. Talár 51
6.22. Talár hatására mező kiválasztása 52
6.23. Talár a múzeumon 52
6.24. Erasmus mezőre lépés után felugró ablak 53
6.25. Erasmus - mező kiválasztása 54
6.26. Adózás 55
6.27. Esély kártya 55
6.28. Dobás küldése websocketen 58
6.29. 1. visszajelzés 61
6.30. 2. visszajelzés 61
6.31. 3. visszajelzés 62
6.32. 4. visszajelzés 62
6.33. 5. visszajelzés 63
6.34. 6. visszajelzés 63
6.35. 7. visszajelzés 64
6.36. 8. visszajelzés 64

Táblázatok jegyzéke

4.1. Városok és csoportjaik 20

6.1. Szobarendszerhez tartozó eventek 37
6.2. Játékhoz tartozó eventek 57